
存储设备及其对 底层系统设计的影响



崔立骁

cuilx@nbjl.nankai.edu.cn

Slides部分来源于Animesh Trivedi,
Storage Systems (StoSys)
XM_0092, 以及清华大学存储系统课程



Parallel and Distributed
Software Technology Lab



Nankai - Baidu
Joint Laboratory



南开计算机系统前沿 2026...
南开大学



仅限企业内部成员加入

该二维码 1 年内 (2027 年 2 月 24 日前) 有效



课前声明：本次课程涉及大量硬件和底层系统知识，你可能会感到有些无趣。后续课程会“自底向上”讲解更上层系统设计

应用层

- 面向机器学习的存储系统
- 向量存储系统
- 分布式存储
- 持久内存存储系统
- 键值对存储系统
- 文件系统
- 存储设备及对底层系统设计的影响

硬件层



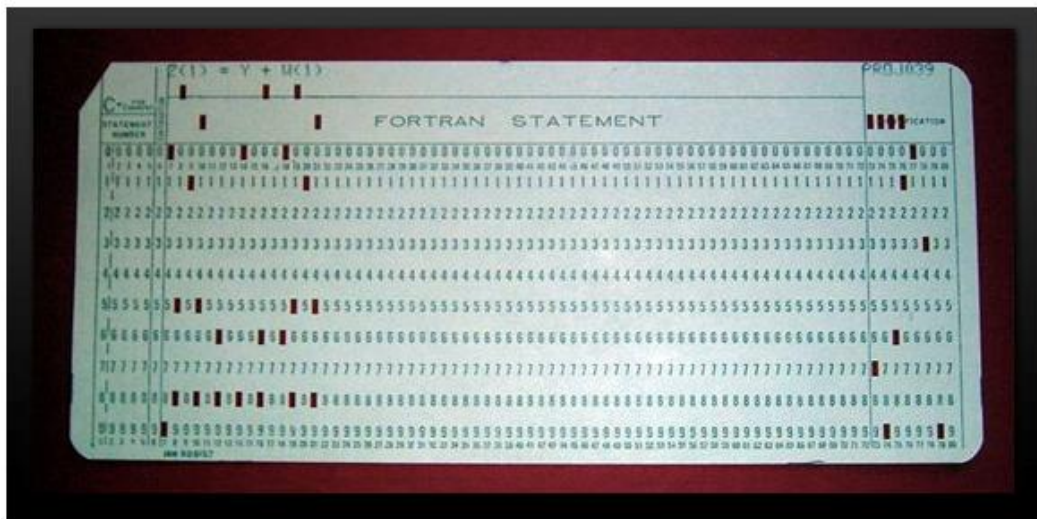


目录

- 存储硬件介绍
- 存储性能的度量方式
- 存储硬件对系统设计的影响



存储设备的发展

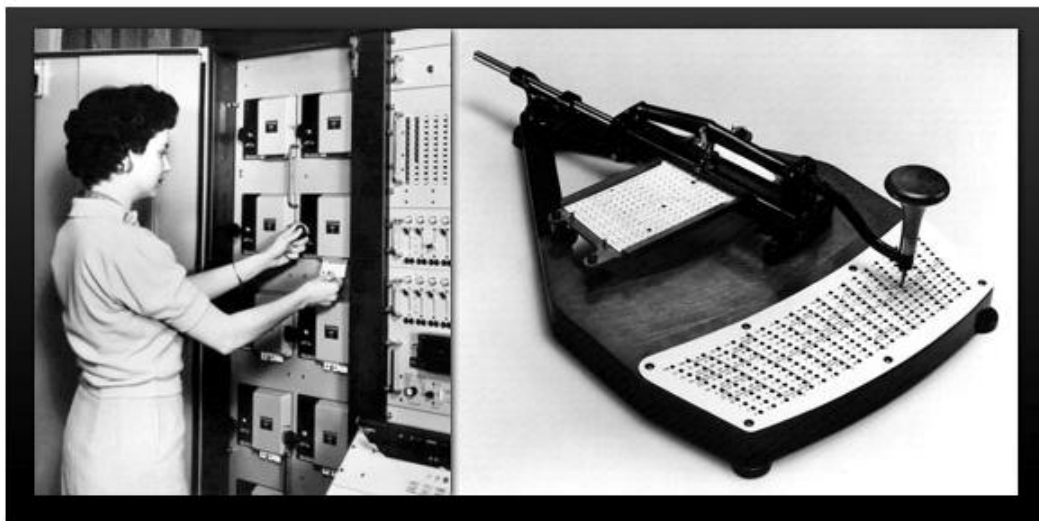


穿孔卡

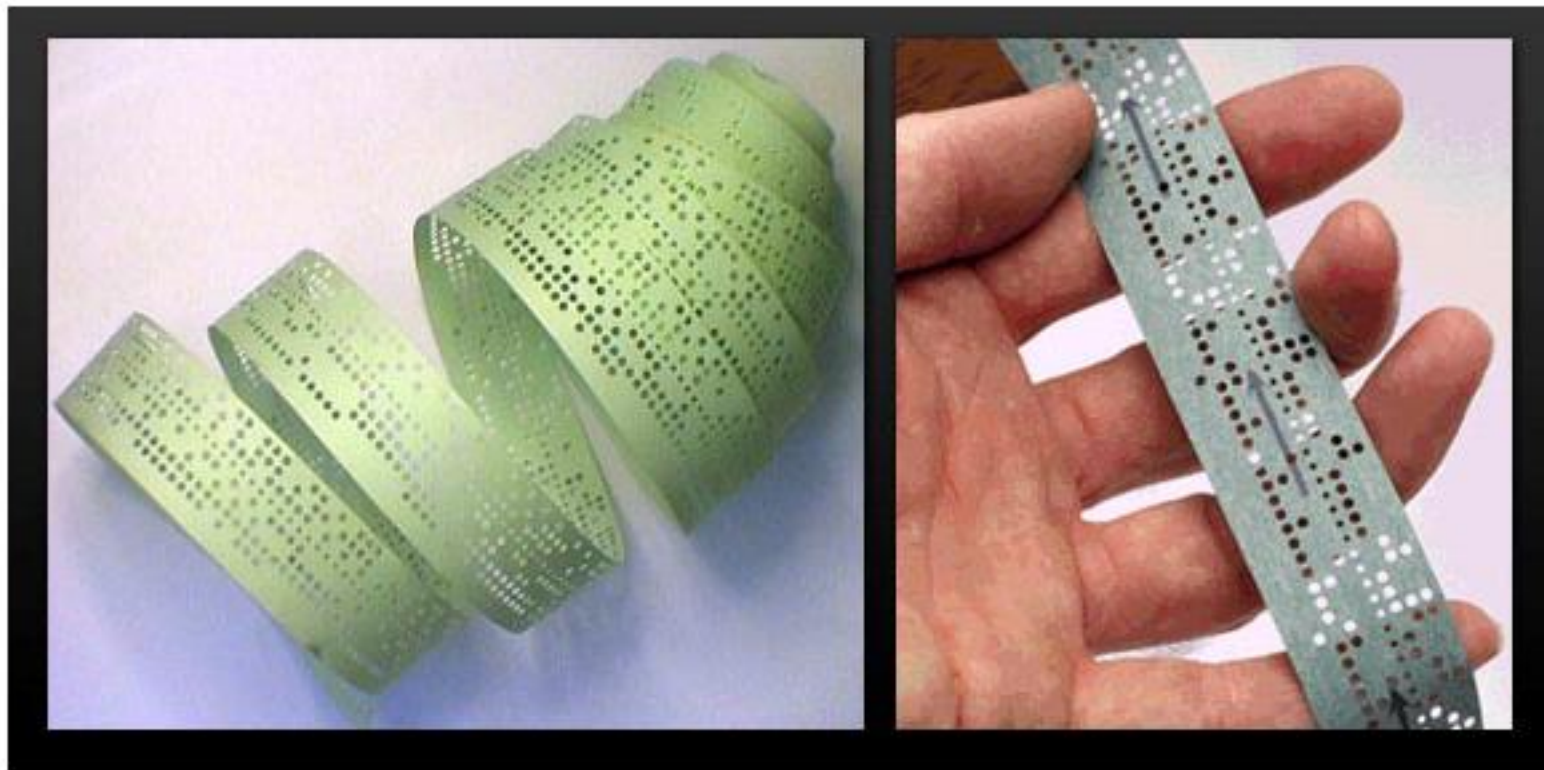
(一个简单的fortran程序)

早在1725用于机器，1970年代广泛用于计算机。

穿孔卡的Reader 和
穿孔卡Writer



存储设备的发展



穿孔带

存储设备的发展



磁带 (Magnetic Tape)

- 左图 1951年 UNIVAC I 计算机
- 右图 1971年 IBM 3410 Magnetic Tape Subsystem

70~80年代用于个人电脑 (PC) 的小型磁带。

- 速度约 250 Byte/s
- 大小单面约660 KB。

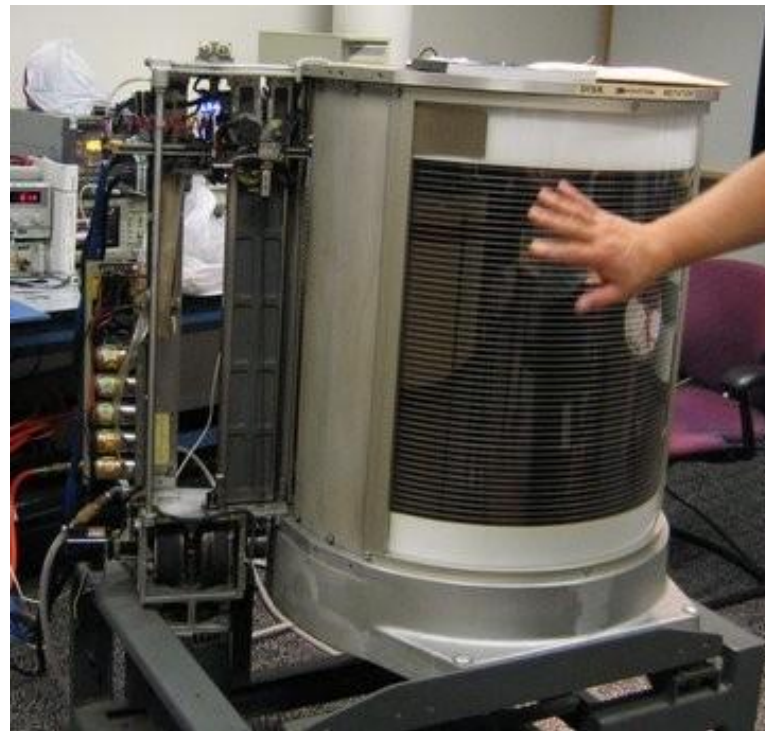


存储设备的发展



磁带

存储设备的发展



硬盘 (Hard Disk Drive)

1956年, IBM的世界上第一块硬盘, 容量5MB

存储设备的发展



软盘 70到90年代广泛使用

存储设备的发展



光盘 1978年面世

存储设备的发展



SSD (Solid State Driver)

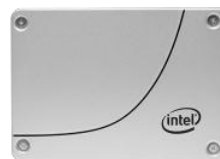
现在流行的存储设备...



Tape



HDD



SATA SSD



NVMe SSD



新兴存储设备...



Tape



SMR HDD



HDD



SATA SSD



NVMe SSD



Persistent Memory

← 备份数据 成本更低

→ 在线数据 性能更高



目录

- 存储设备
- 存储I/O性能
- 存储设备对系统设计的影响



存储性能指标



延迟 (Latency) 完成一次操作的时间。

带宽 (Bandwidth) 读/写速度，通常单位为MB/s 或 GB/s。

IOPS (I/O operations per second) 每秒的操作数。

DRAM
内存



持久型内存



HDD/SSD
存储



延迟

~100 ns

~250 ns

10us ~ 10ms

单条读带宽

~20 GB/s

~ 7 GB/s

120 MB ~ 2.5 GB/s

单条写带宽

~13.5 GB/s

~ 2.3 GB/s

80 MB ~ 2.2 GB/s

存储性能 -- fio测试延迟



(最小)延迟 (Latency) 完成一次操作的时间。

fio 读延迟脚本:

```
1  [global]
2  filename=fio_testdata
3  size=20G
4  runtime=30
5  ioengine=libaio
6  direct=1
7  ramp_time=10          # start measuring after warm-up time
8
9  [read]
10 readwrite=randread
11 blocksize=4K
```

存储性能 -- fio测试带宽



带宽 (Bandwidth) 读/写速度，通常单位为MB/s 或 GB/s。

fio 读带宽脚本：

```
1  [global]
2  filename=fio_testdata
3  size=20G
4  runtime=30
5  ioengine=libaio
6  direct=1
7  ramp_time=10          # start measuring after warm-up time
8
9  [read]
10 readwrite=read
11 numjobs=16
12 blocksize=64k
13 offset_increment=128m  # each job starts at a different offset
```



存储性能 -- fio测试IOPS



IOPS (I/O operations per second) 每秒的操作数。

fio 读IOPS脚本:

```
1  [global]
2  filename=fio_testdata
3  size=20G
4  runtime=30
5  ioengine=libaio
6  direct=1
7  ramp_time=10          # start measuring after warm-up time
8
9  [read]
10 readwrite=randread
11 numjobs=64
12 blocksize=4K
```



存储性能指标

- **延迟 (Latency)** 完成一次操作的时间。
测试方法: 小块(4KB), 单线程
- **带宽 (Bandwidth)** 读/写速度。
测试方法: 大块(64KB~8MB), 单/多线程
- **IOPS (I/O operations per second)** 每秒的操作数。
测试方法: 小块(4KB), 多线程

DRAM
内存



持久型内存



HDD/SSD
存储



	DRAM 内存	持久型内存	HDD/SSD 存储
延迟	~100 ns	~250 ns	10us ~ 10ms
单条读带宽	~20 GB/s	~ 7 GB/s	120 MB ~ 2.5 GB/s
单条写带宽	~13.5 GB/s	~ 2.3 GB/s	80 MB ~ 2.2 GB/s



存储性能指标 – 延迟-吞吐曲线

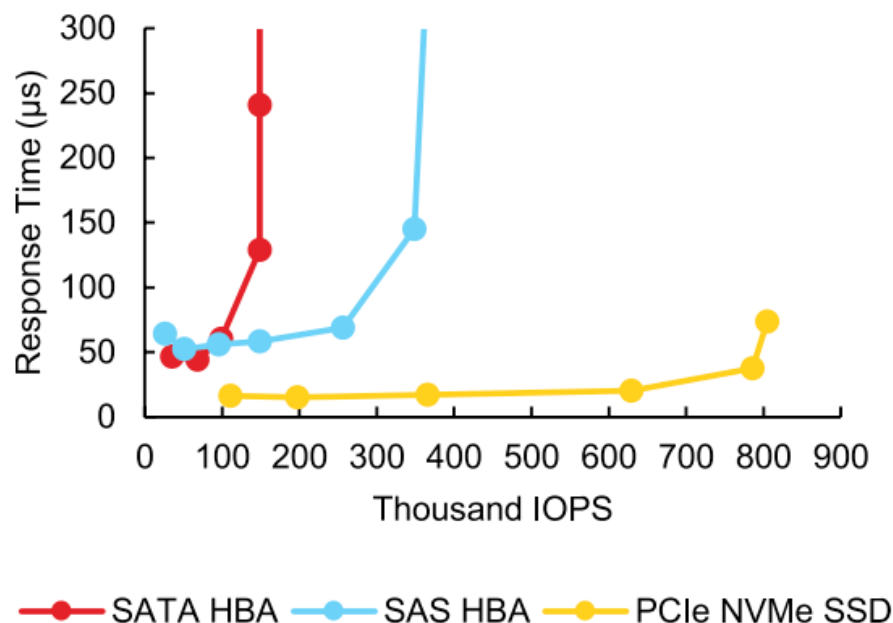


Fig. 3. 99.99th percentile response times (excluding media) plotted against IOPS for 4-kB reads for three SSD interfaces.

三种设备的延迟-吞吐曲线 [8]， 可以发现在接近吞吐极限时延迟也急剧增加。所以对于延迟敏感的上层的在线应用，一定要考虑IO负载，尽量不要打满。



目录

- 存储设备
- 存储I/O性能
- 存储设备对系统设计的影响
 - 存储硬件内部的变化
 - 硬件存储栈
 - 软件存储栈



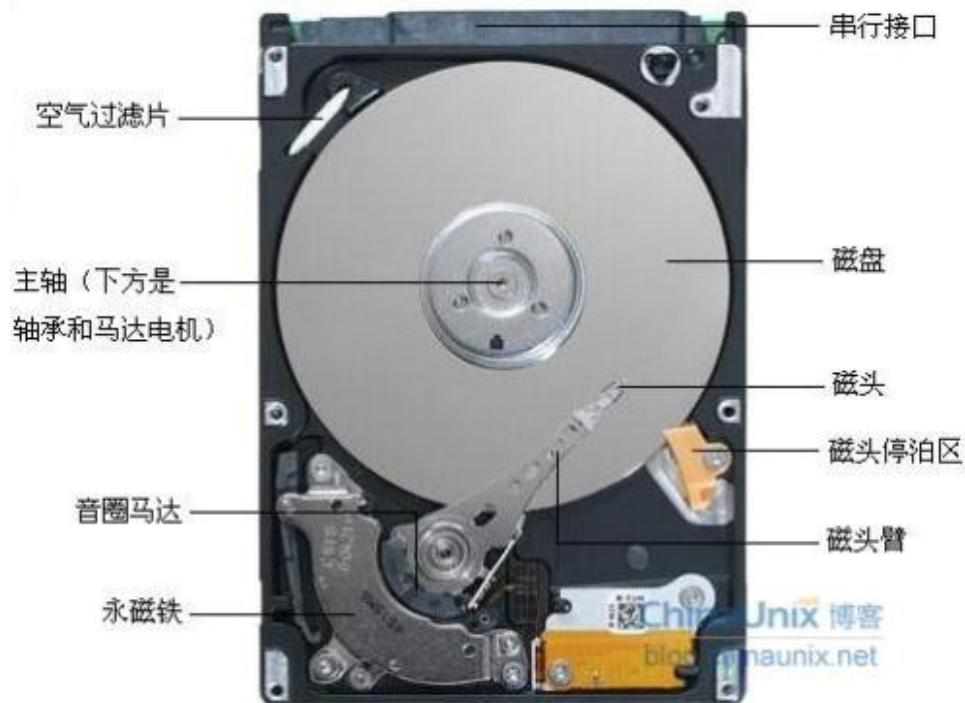
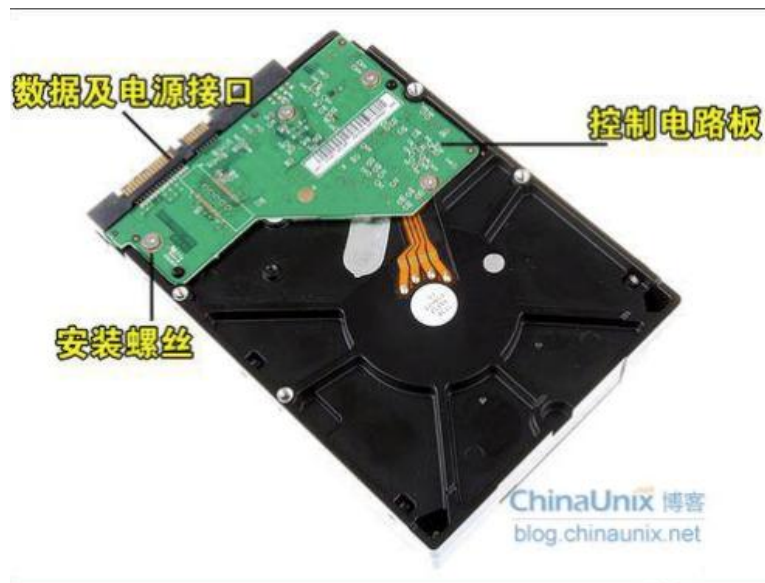


目录

- 存储设备
- 存储I/O性能
- 存储设备对系统设计的影响
 - **存储硬件内部的变化**
 - 硬件存储栈
 - 软件存储栈

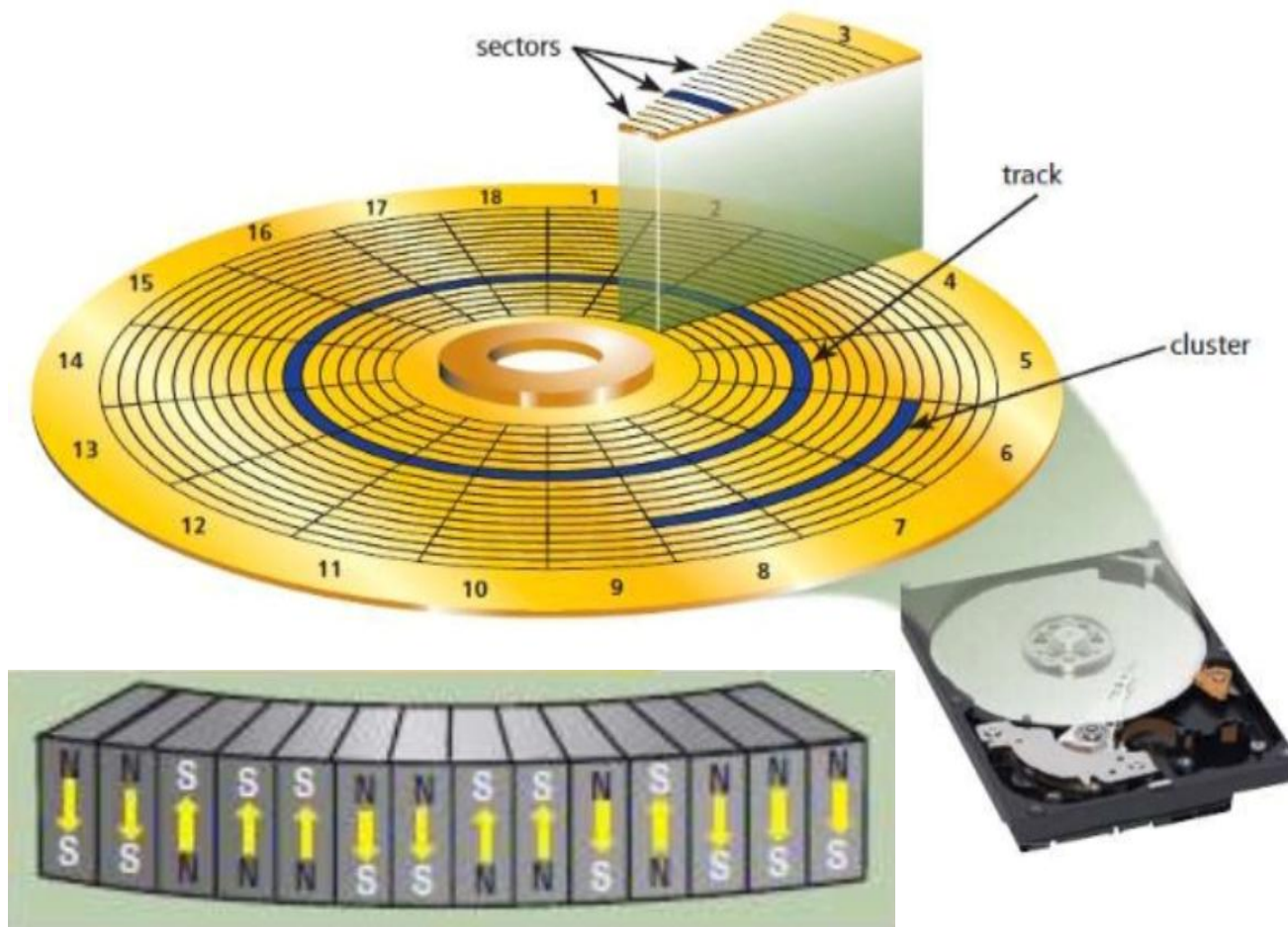


HDD 磁盘存储原理



内部构造

HDD 磁盘存储原理



存储原理



HDD 磁盘I/O性能

- 两种HDD的随机/顺序:

	(SCSI盘) Cheetah	(SATA盘) Barracuda
$R_{I/O}$ Random	0.66 MB/s	0.31 MB/s
$R_{I/O}$ Sequential	125 MB/s	105 MB/s

HDD随机读写性能很差!

- HDD随机读写延迟模型:

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

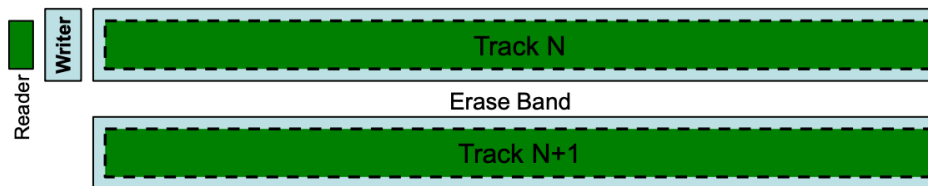
- 随机读写情况下:

$$T_{seek} = 4 \text{ ms}, T_{rotation} = 2 \text{ ms}, T_{transfer} = 30 \text{ microsecs}$$

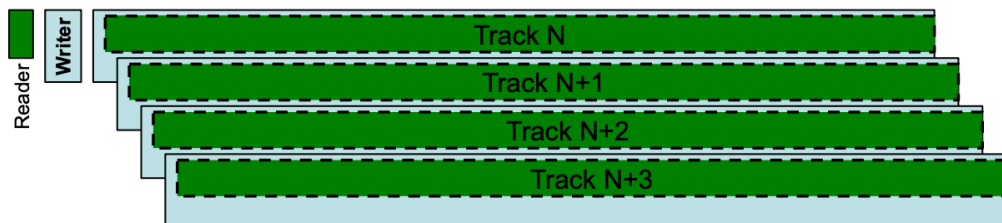


SMR HDD 磁盘存储原理

Conventional Writes



SMR Writes



叠瓦式磁记录 HDD

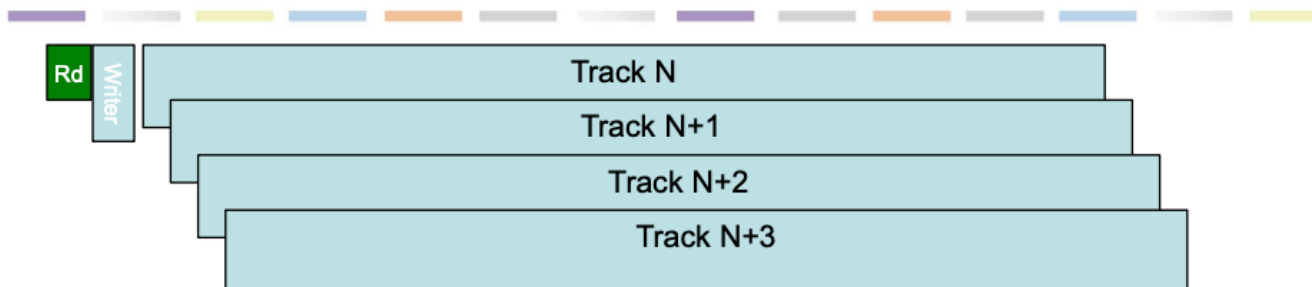
Shingled Magnetic Recording (SMR) HDD

- 存储密度
- 随机读写性能更差

SMR HDD 磁盘存储原理



Updating a band with new data



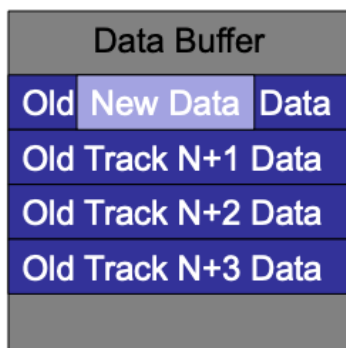
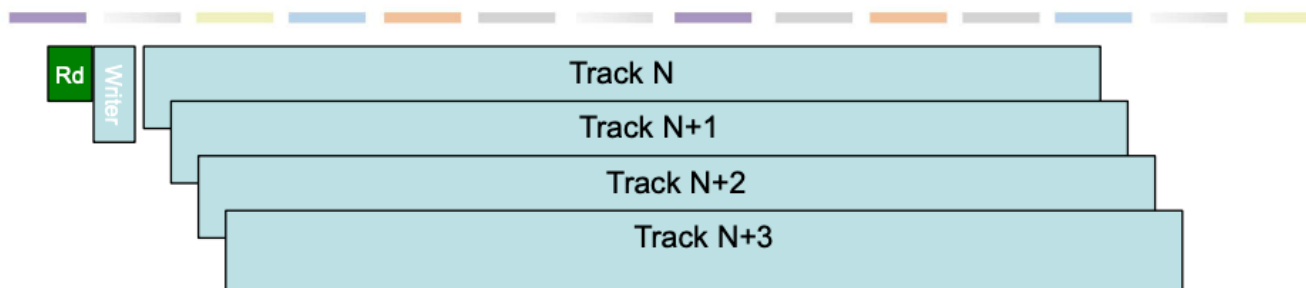
Data Buffer	
Old Track N	Data
Old Track N+1	Data
Old Track N+2	Data
Old Track N+3	Data

1. Read old data

SMR HDD 磁盘存储原理



Updating a band with new data



1. Read old data
2. Merge with new data

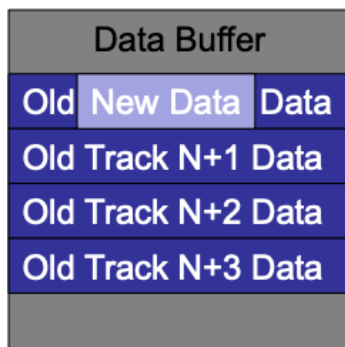
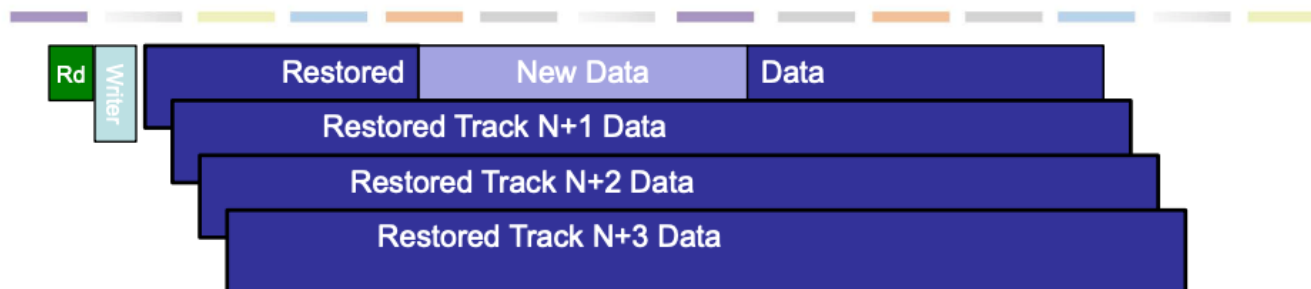
Shingled Magnetic Recording – Models, Standardization, and Applications
© 2014 Storage Networking Industry Association. All Rights Reserved.

17

SMR HDD 磁盘存储原理



Updating a band with new data



1. Read old data
2. Merge with new data
3. Write new data, refreshing old data

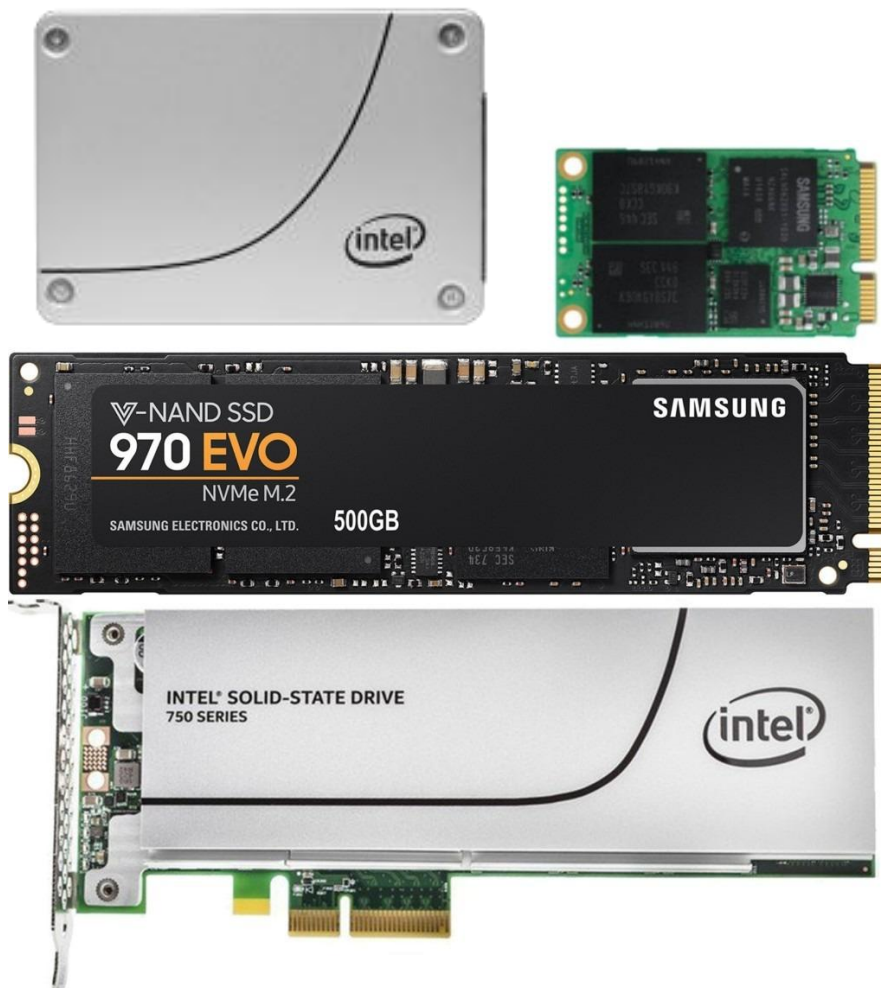
Shingled Magnetic Recording – Models, Standardization, and Applications
© 2014 Storage Networking Industry Association. All Rights Reserved.



HDD 和 SMR HDD I/O性能优化

- **更大的读写单元：**减小随机性，充分利用顺序读写带宽
- **内存作磁盘cache：**利用数据访问的空间/时间局部性，用更快的内存加快IO。
- **IO合并：**将多次空间连续但是提交时间乱序的小IO合为一次连续大IO。
- **使用Log structured 方式优化写性能：**用追加的方式将随机写转为顺序写。

Nand Flash based SSD



Nand Flash 存储介质

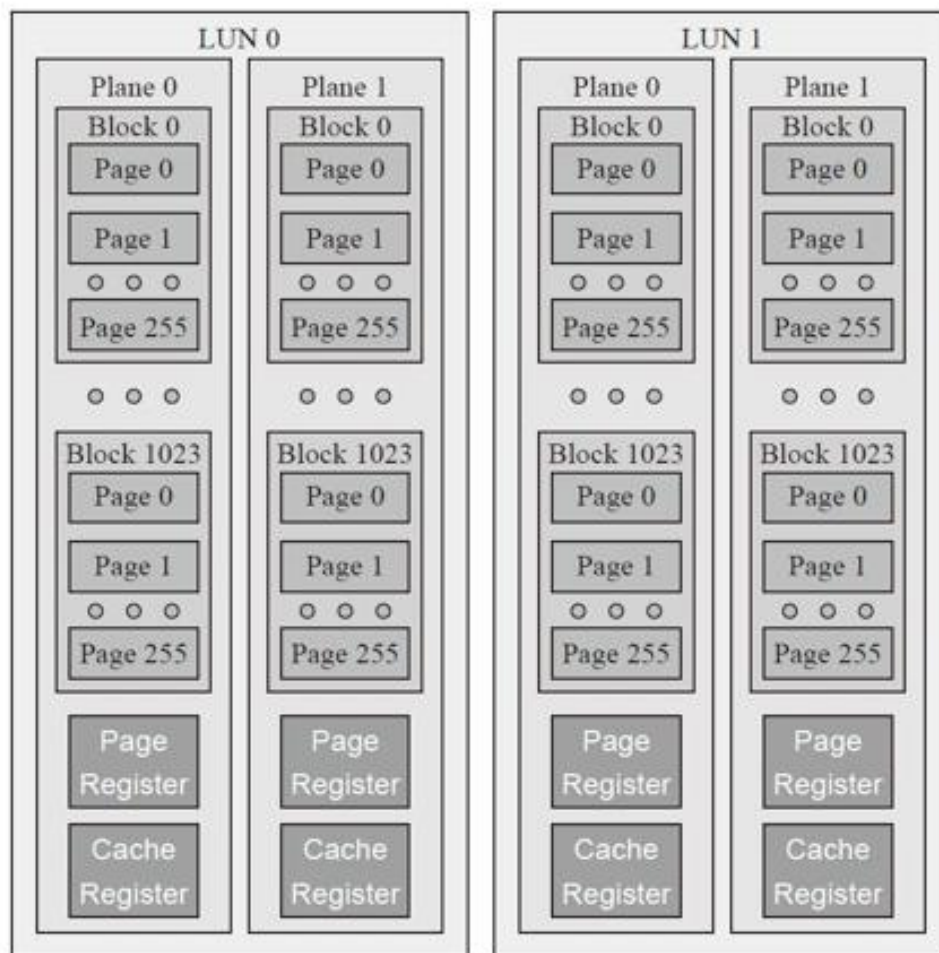


图3-7 闪存内部组织架构

Nand Flash 存储介质

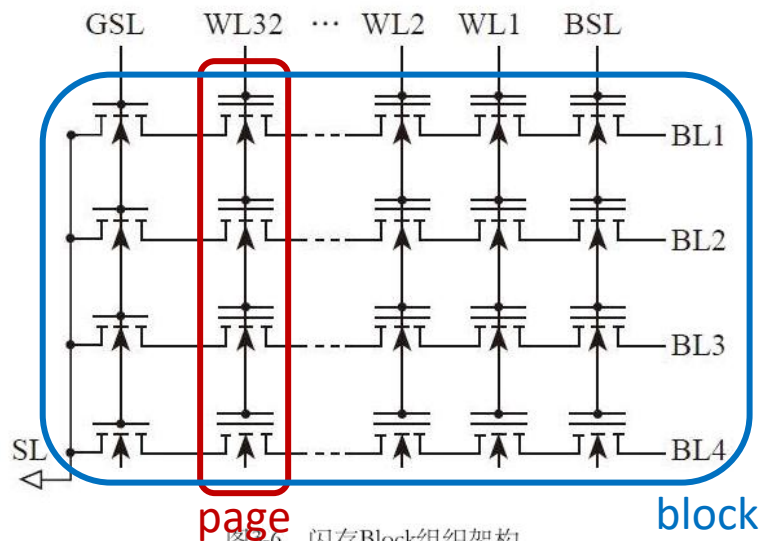


图2-6 闪存Block组织架构

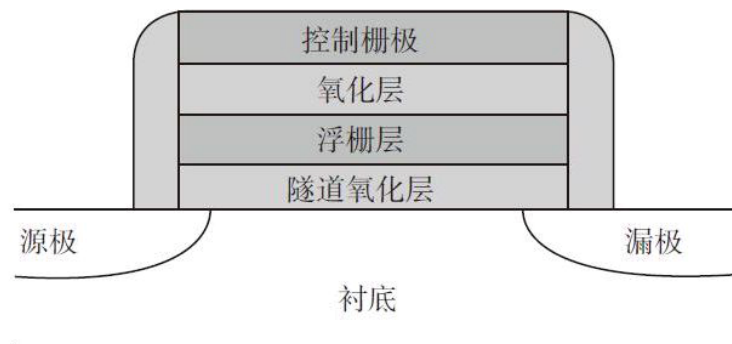


图3-1 浮栅晶体管结构

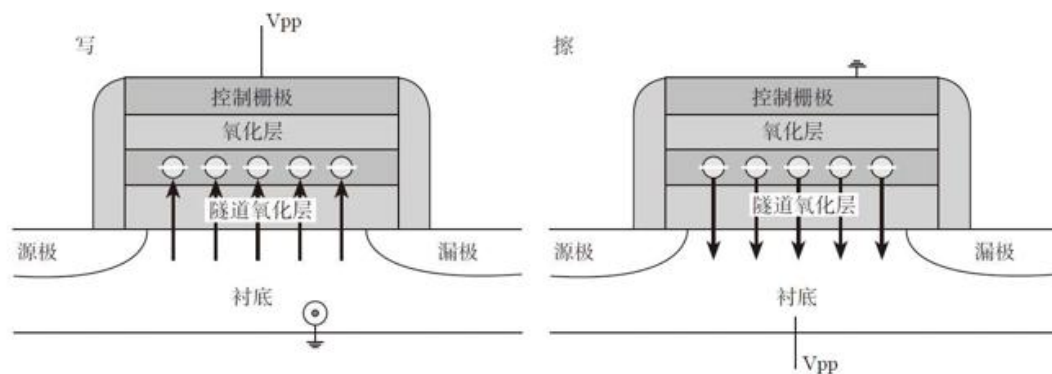


图3-2 左：写原理；右：擦除原理

写：控制栅极加正电，衬底接地，电子被“吸入”浮栅层。（写为0）

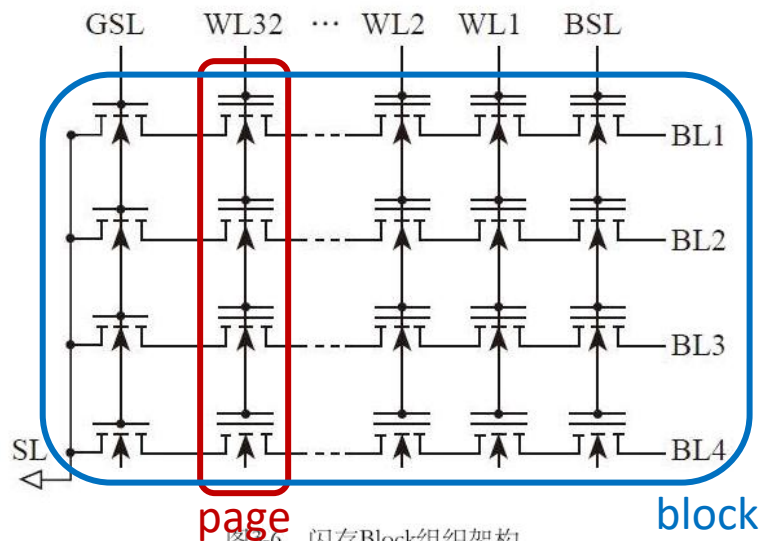
擦：衬底(P-well)加正电，电子被“吸”回来，数据被擦除。（擦为1）

读：以page为单位，要读的page栅极加电压，通过读BL来确定0 / 1值。

注意：衬底加电压整个block会被擦，所以并非字节寻址。



Flash 作存储的复杂性



- 最小读写单元为page。
- 擦除后的page只能写一次，要覆盖写page需要先擦除，擦除单位为block。
- 每个block存在最大擦除次数。

- 由于Block size大于page size，所以可能需要进行“读-修改-写操作” (Read-Modify-Write, RMW) 造成**写放大问题**。Over-provision(OP)和提前的垃圾回收(GC)可以减少SSD写放大!
- 由于每个block存在最大擦除次数，即使用寿命，所以需要**磨损均衡**。



管理的Nand Flash的SSD FTL层

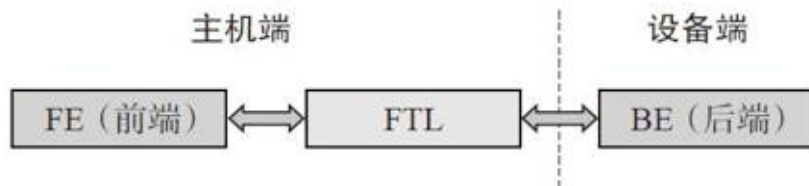


图4-1 FTL在主机端

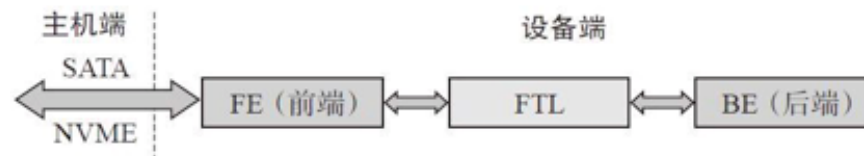


图4-2 FTL在设备端

一般实现于设备端，主要功能：

- 管理逻辑块-物理块的映射
- 负责垃圾回收(GC)和over-provision(OP)
- 磨损平衡

SSD FTL层的OP



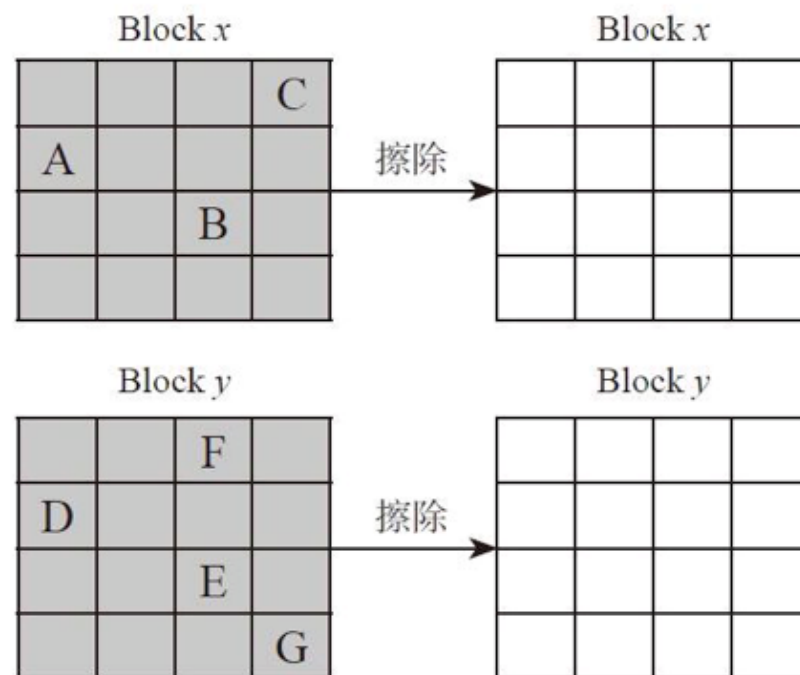
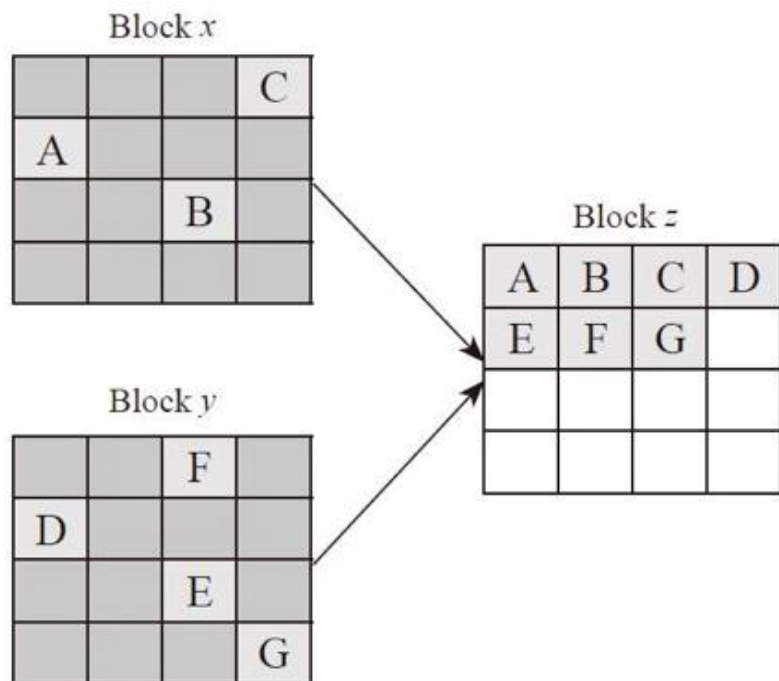
	CH0			CH1			CH2			CH3		
Block 0	1	5	9	2	6	10	3	7	11	4	8	12
	13	17	21	14	18	22	15	19	23	16	20	24
	25	29	33	26	30	34	27	31	35	28	32	36
	37	41	45	38	42	46	39	43	47	40	44	48
Block 1	49	53	57	50	54	58	51	55	59	52	56	60
	61	65	69	62	66	70	63	67	71	64	68	72
	73	77	81	74	78	82	75	79	83	76	80	84
Block 2	85	89	93	86	90	94	87	91	95	88	92	96
	97	101	105	98	102	106	99	103	107	100	104	108
	109	113	117	110	114	118	111	115	119	112	116	120
Block 3	121	125	129	122	126	130	123	127	131	124	128	132
	133	137	141	134	138	142	135	139	143	136	140	144
	145	149	153	146	150	154	147	151	155	148	152	156
Block 4	157	161	165	158	162	166	159	163	167	160	164	168
	169	173	177	170	174	178	171	175	179	172	176	180
Block 5												

考虑覆盖写block 0 中的某一page:

1) Block 5不作OP, 暴露给主机:
需要读block 0中 9 个page到buffer,
修改其中1个page, 擦除block 0,
再写入9页, 造成了“写放大”。

2) Block 5 作OP, 不暴露给主机:
可以直接写Block 5

SSD FTL层的GC



SSD 的 I/O性能



Intel SSD 750 参数		
容量	400GB	1.2TB
外形尺寸	2.5"15mm SFF-8639 or PCIe Add-In Card (HHHL)	
接口	PCIe3.0 × 4-NVMe	
控制器	Intel CH29AE41AB0	
NAND	Intel 20nm 128Gbit MLC	
顺序读	2200MB/s	2400MB/s
顺序写	900MB/s	1200MB/s
4KB 随机读	430k IOPS	440k IOPS
4KB 随机写	230k IOPS	290k IOPS
空闲功耗	4W	4W
读写功耗	9W/12W	10W/22W
加密支持	N/A	
耐写性	70GB Writes per Day for Five Years	

SSD不再适用传统的HDD优化



- **更大的读写单元：**~~减小随机性，充分利用顺序读写带宽~~ (可以用更小读写单元)
- **内存作磁盘cache：**利用数据访问的空间/时间局部性，用更快的内存加快IO。(作用减弱)
- **IO合并：**~~将多次空间连续但是提交时间乱序的小IO合为一次连续大IO。~~ (反而增加延迟，越高性能SSD越不需要合并)
- **使用Log structured 方式优化写性能：**用追加的方式将随机写转为顺序写。(意外收获：用在SSD上可以减少写放大)

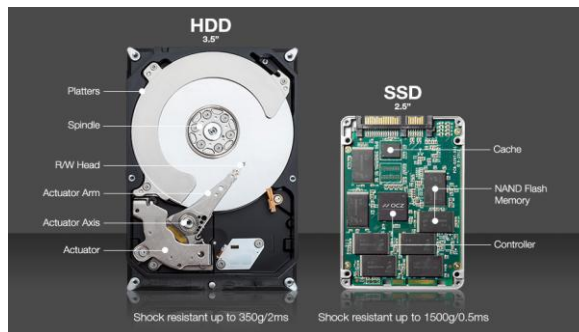


目录

- 存储设备
- 存储I/O性能
- 存储设备对系统设计的影响
 - 存储硬件内部的变化
 - **硬件存储栈**
 - 软件存储栈

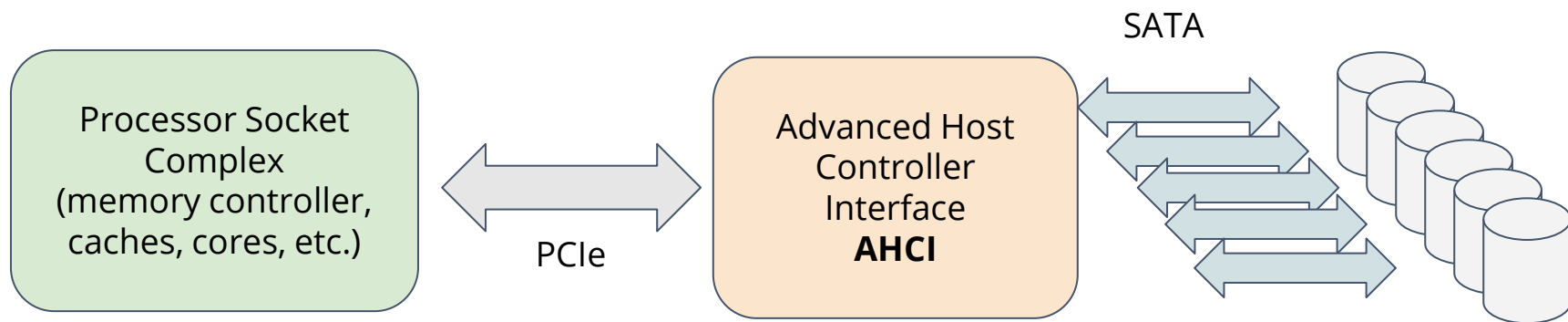


SSD硬件协议



- 主流计算领域的首批闪存设备出现在2000年代中期（2005-2006-2007年）
- 闪存NAND/NOR目前已经被广泛使用。
- 通常新技术会先复用已有的系统接口
- 第一代闪存设备包装为快速“硬盘”，支持兼容的SAS/SATA硬盘协议用于数据传输

经典硬盘接口：AHCI

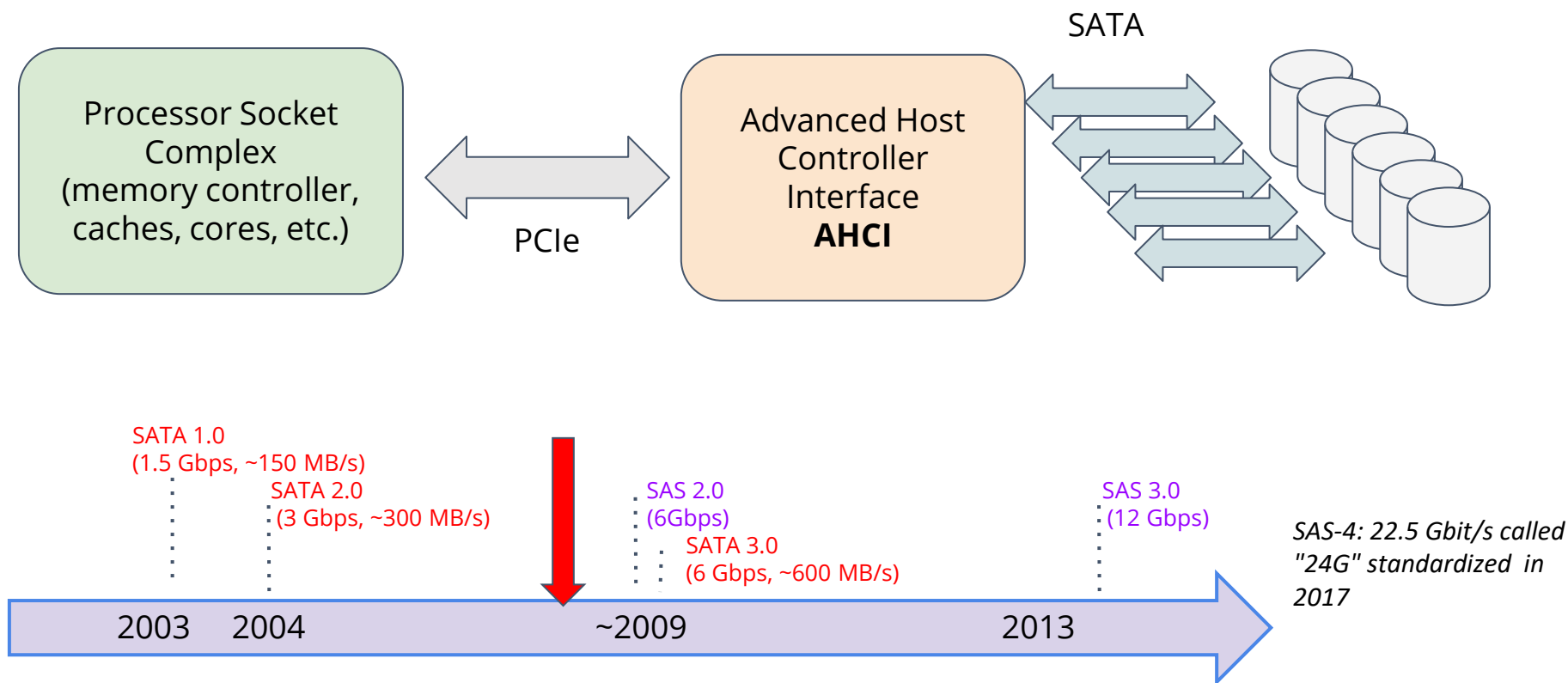


- AHCI被应用于主板上
- 实现PCIe与SATA之间的**单点**协议转换
- 支持多种设备类型（硬盘、光驱、软盘驱动器）

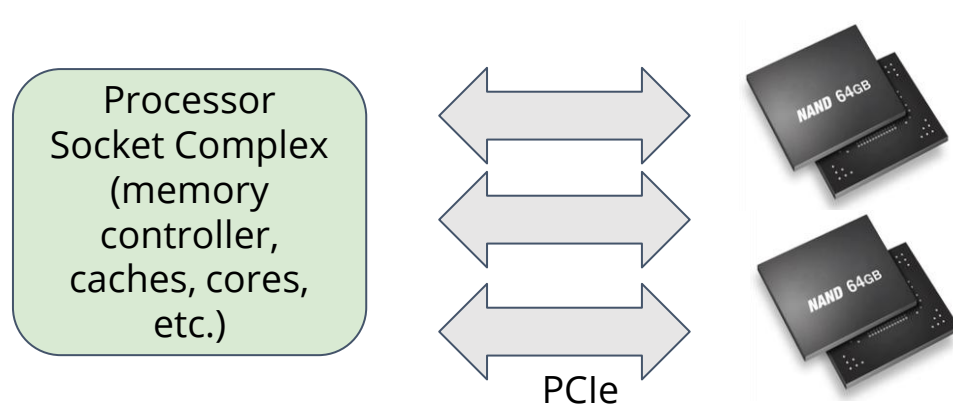
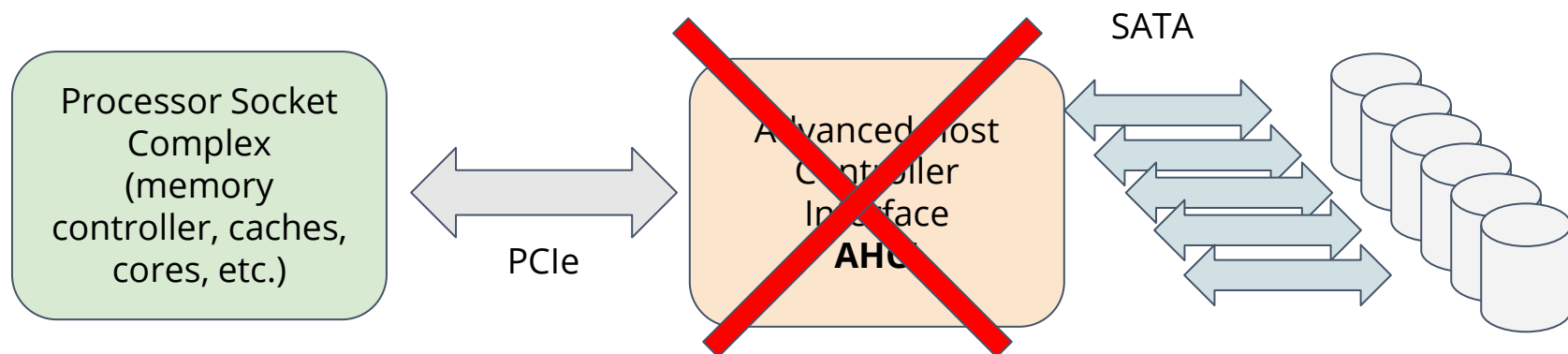


AHCI在SSD时代的挑战

单一通道传输导致性能瓶颈，SATA速度也无法充分发挥SSD性能



将高速的NVM SSD连接在PCIe总线



为什么要直接把设备接到PCIe总线上?

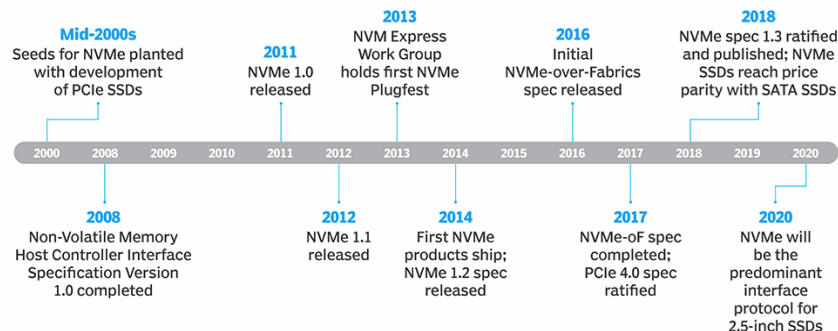
- 没有HBA (总线适配器), 直接连接到CPU。
- 可扩展端口宽度 (1-16x通道)
- 高带宽/通道 (v2.0版本时 ~500MB/通道, 现今 5.0 为 ~4GB/通道)
- 标准总线, 所有平台均支持。

NVMe 硬件存储栈



图6-3 NVMe处于协议栈的最高层

NVMe milestones

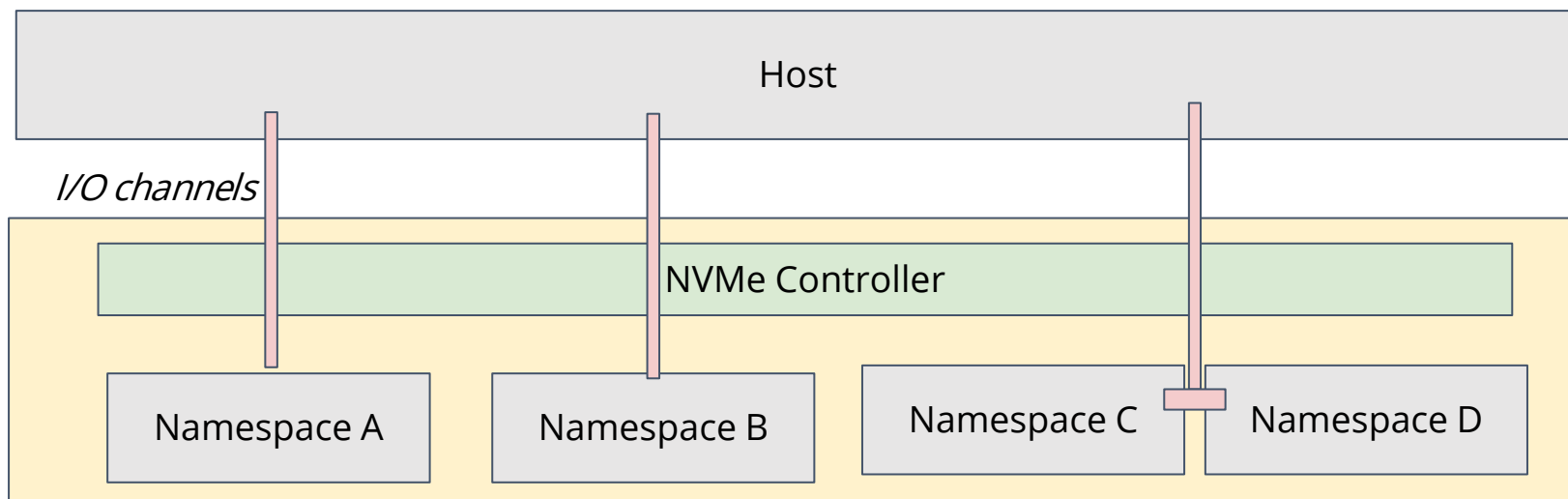


NVMe是一种协议规范，它规定了主机软件如何通过 PCI Express (PCIe) 总线与 NVM SSD 存储设备通信。

NVMe协议 – 命名空间

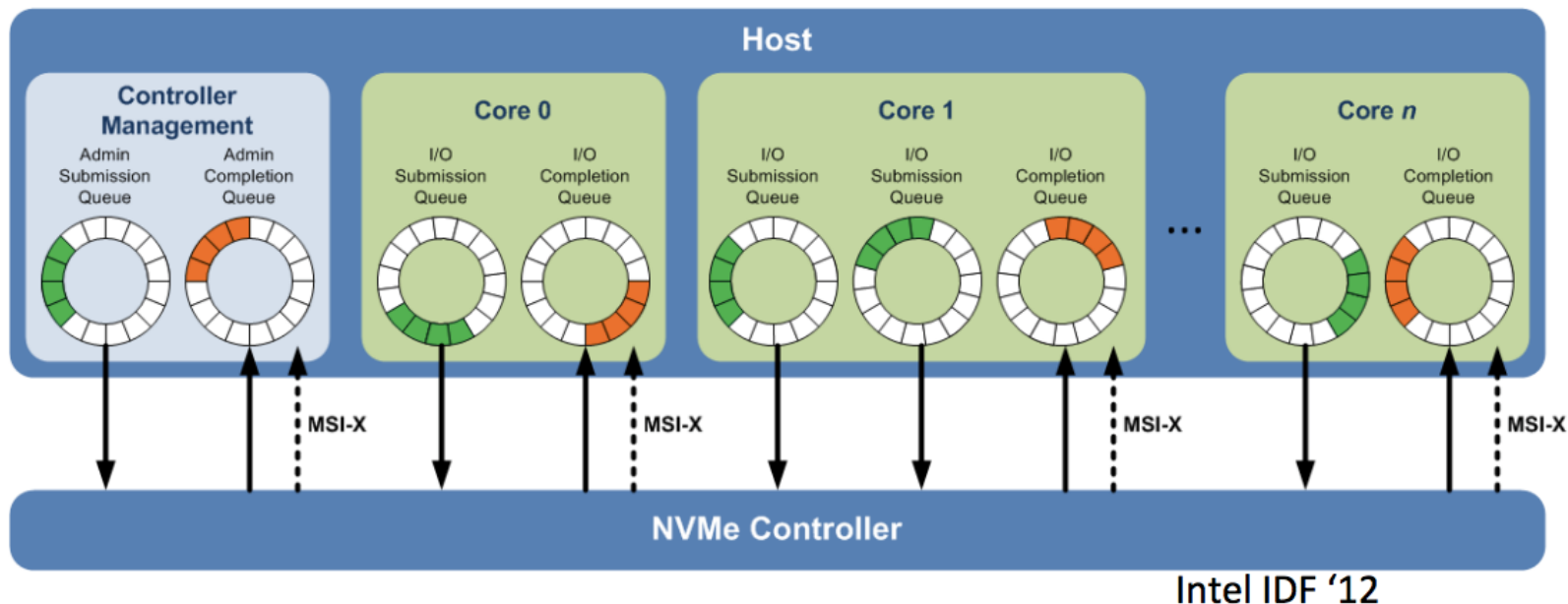


如何利用NVMe SSD内部的并行性



单个设备中可设置多个独立分区，构成多个命名空间
可以在命名空间中创建独立的I/O通道

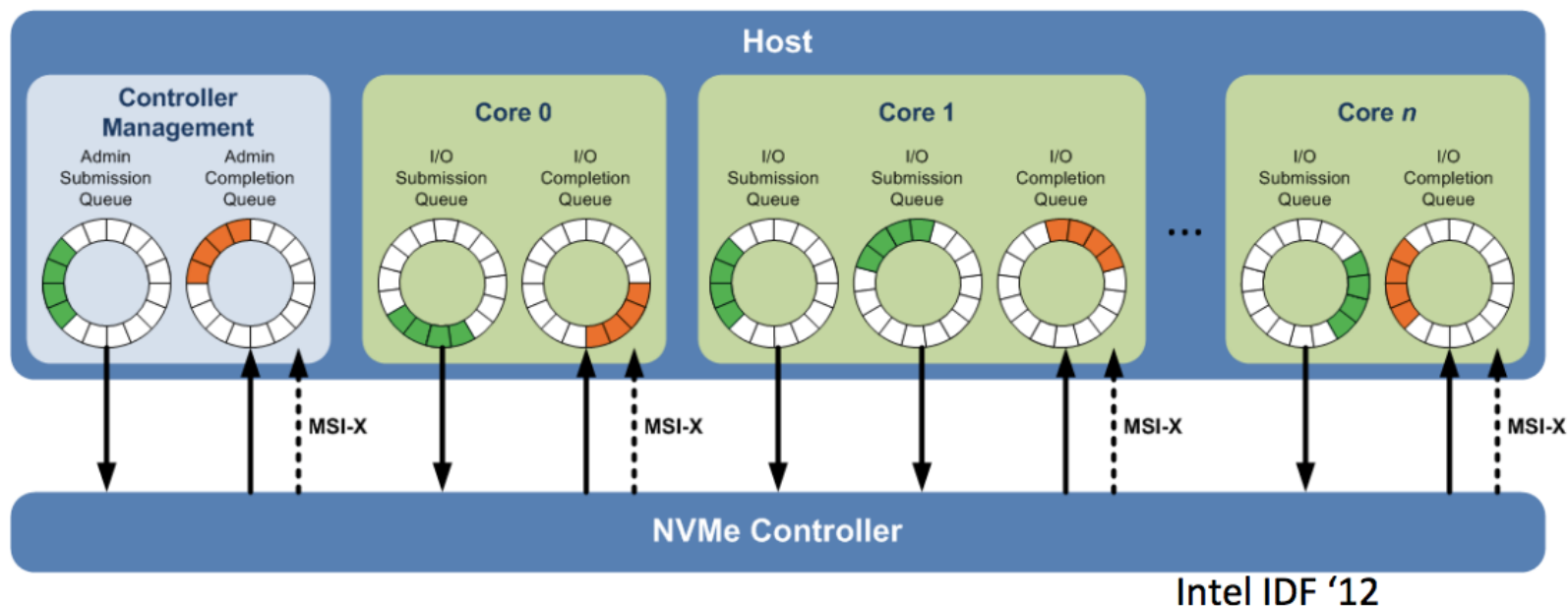
NVMe协议 – 队列



- I/O SQ/CQ可以最多有65535对：
 每个物理核最多1个CQ；
 但每个核可以有多个SQ，因此可以实现线程粒度的不同优先级。
- I/O SQ/CQ队列深度可达到64K。
- Admin SQ/CQ 每个系统只有一对。



硬件存储栈 – NVMe协议



1. 主机写命令到SQ;
2. 主机写SQ的DoorBell, 通知SSD取指;
3. SSD到SQ中取指执行, 并写执行结果到CQ;
4. 然后SSD发(MSI-X)中断通知主机指令完成;
5. 收到中断, 主机处理CQ, 查看指令完成状态;
6. 通过DoorBell回复SSD: 指令执行已完!

(Host写SQ)
(Host通知SSD)
(SSD执行并写CQ)
(SSD通知Host)
(Host处理完成)
(Host通知SSD完成)



NVMe与AHCI的对比

充分利用设备并行机会

- HDD 速度慢，设备内部队列机制提升有限
- NVMe SSD 速度快，且包含大量并行组件，设备内部队列机制有助于提升速度
- NVMe 最大64K 队列，64K队列深度 vs. AHCI 支持32 队列，32队列深度
- NVMe支持多个中断， AHCI 仅支持单个中断

早期NVMe原型

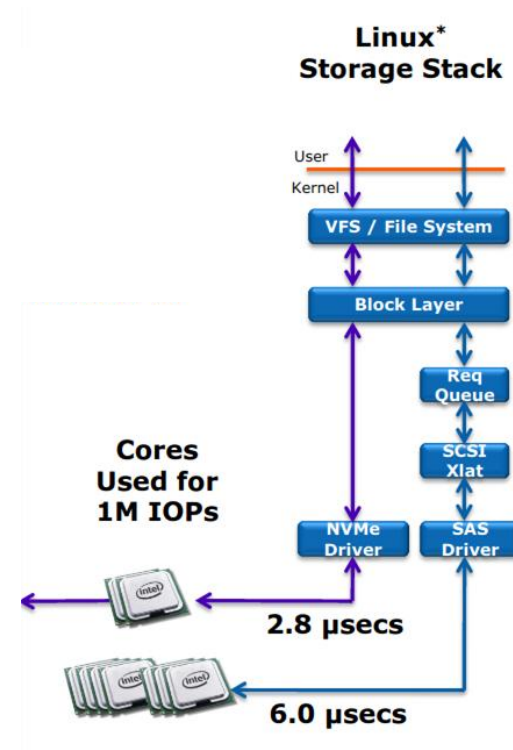
Chatham NVMe Prototype



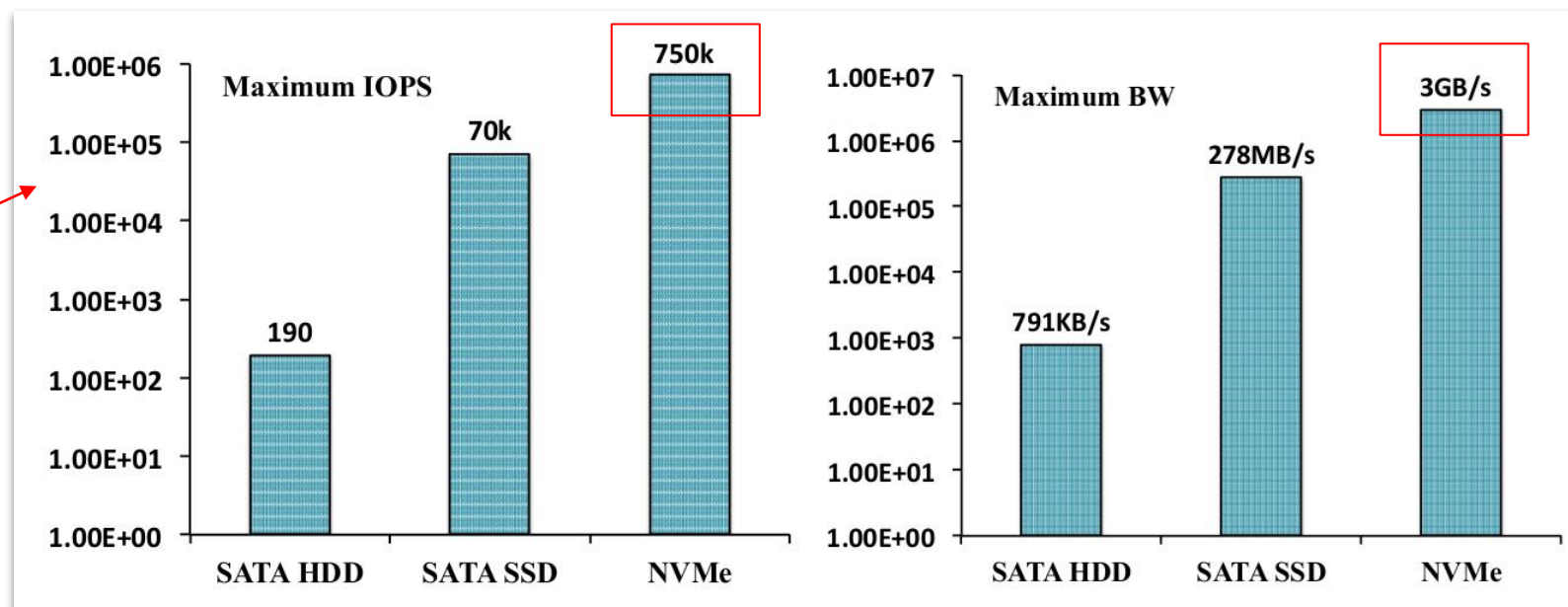
NVMe 降低延迟开销超过50%

- SCSI/SAS : 6.0 μ s 19,500 cycles
- NVMe : 2.8 μ s 9,100 cycles

PCIe 消除了硬件/连接开销
NVMe 消除硬件协议开销



NVMe: 提升IOPS和带宽

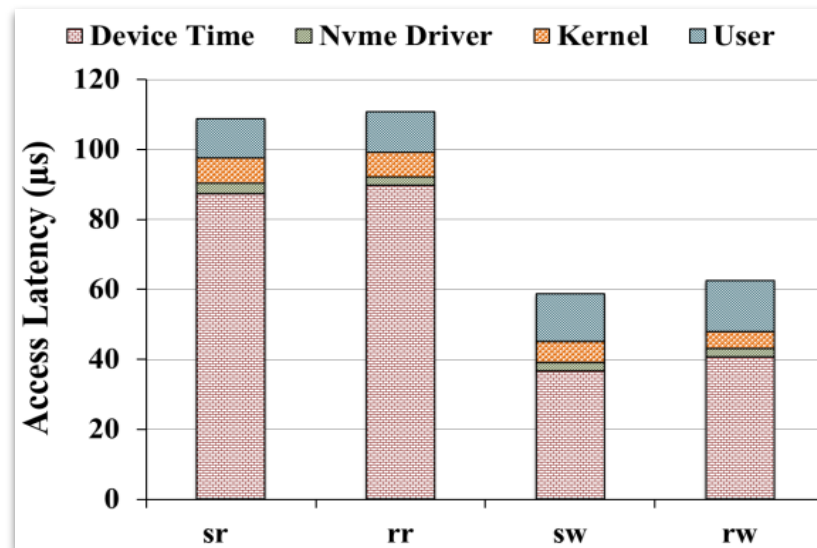
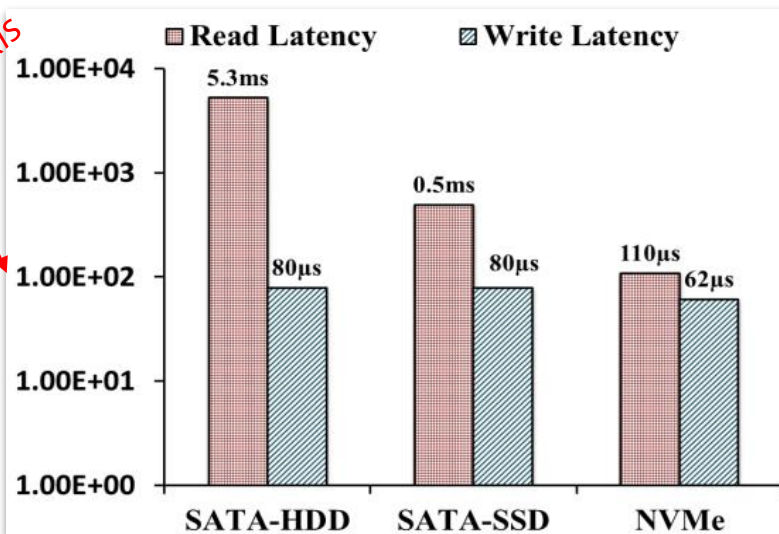


- 单个NVMe设备实现了75万IOPS!
- 3 GB/s带宽 (受PCIe链路限制)

Xu and others, *Performance Analysis of NVMe SSDs and their Implication on Real World Databases*, ACM Syster 2015, <https://dl.acm.org/doi/10.1145/2757667.2757684>.

NVMe: 延迟

Log y axis



相比SATA SSD和HDD，读取性能提升1到3个数量级
随机和顺序读写延迟较为接近

Xu and others, *Performance Analysis of NVMe SSDs and their Implication on Real World Databases*, ACM Syster 2015, <https://dl.acm.org/doi/10.1145/2757667.2757684>.



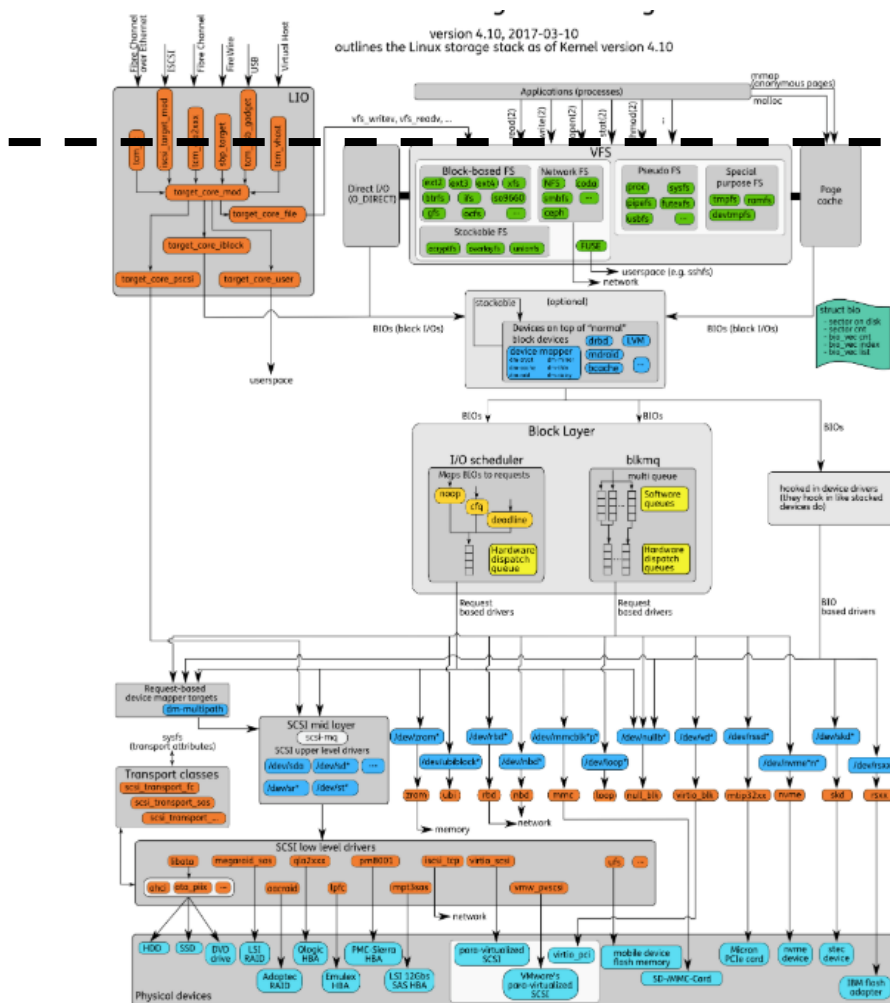
目录

- 存储设备
- 存储I/O性能
- 存储设备对系统设计的影响
 - 存储硬件内部的变化
 - 硬件存储栈
 - **软件存储栈**





Linux 内核存储栈



应用层

(用户态)

文件系统层

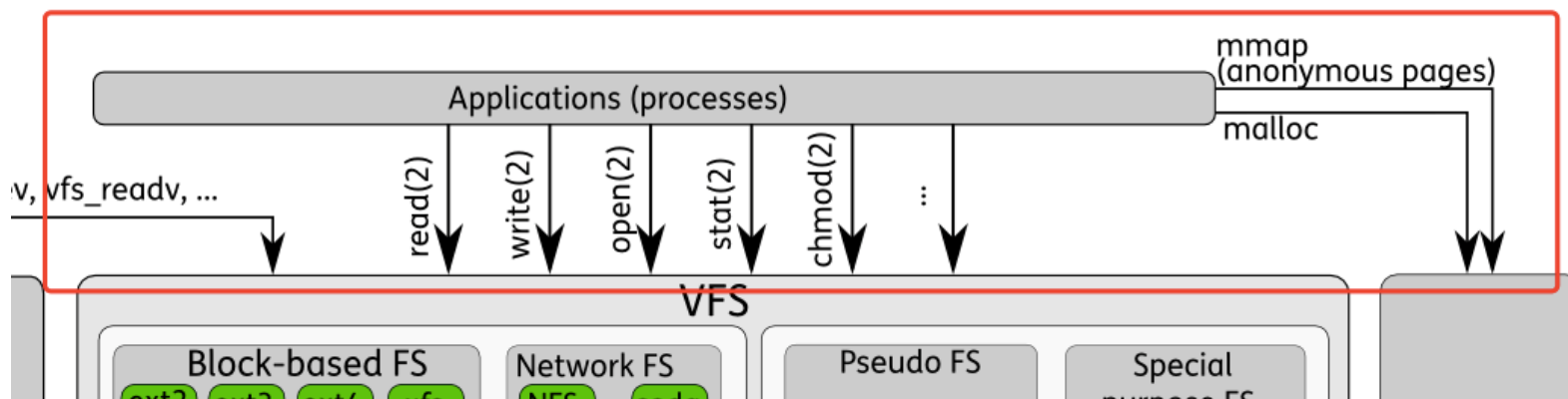
(内核态)

块IO层

设备驱动层



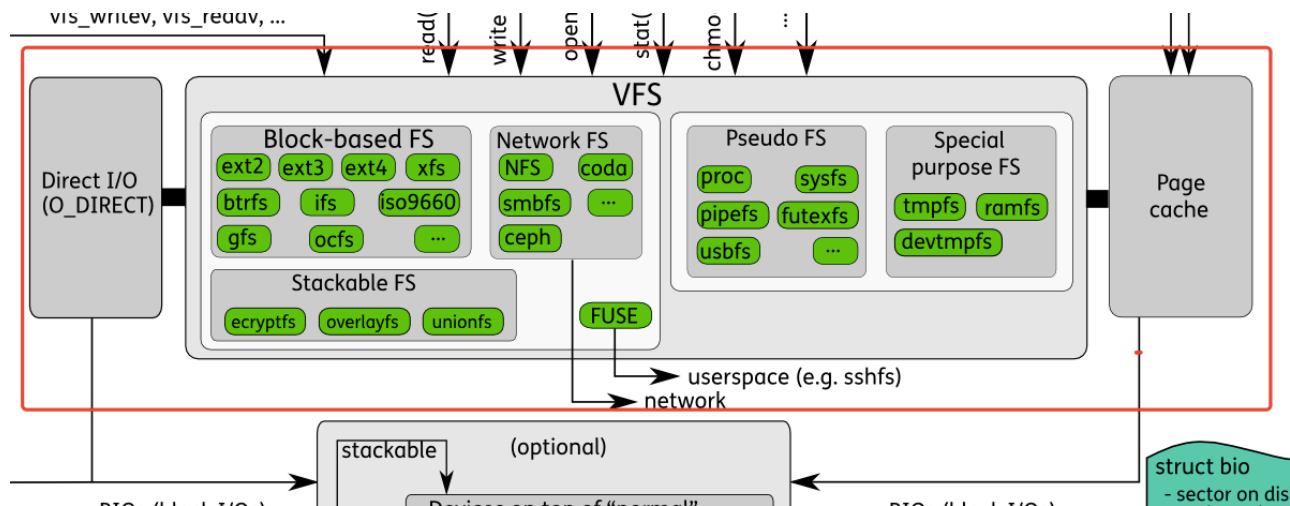
Linux 内核存储栈 -- 应用层



最不固定(灵活), 但也是存储栈重要一环, 比如数据库存储引擎一般实现于应用层, 最终通过系统调用进入文件系统层。



Linux 内核存储栈 -- 文件系统层



- VFS为不同文件系统的实现提供统一的接口，调用对应的文件系统实现。
- 主要分两个路径Buffered I/O (使用Page Cache)和Direct I/O (open文件时加O_DIRECT标志)。



文件系统层 -- 作用和功能

文件和文件夹(目录)形式的管理是最为广泛使用的数据存储抽象。文件系统负责管理文件数据的存取。

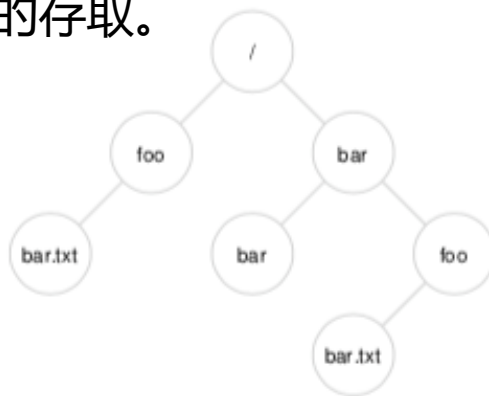


Figure 39.1: An Example Directory Tree

一些文件系统的常用接口：

创建删除文件：

creat, unlink

打开文件：

open

读写文件：

read, write

映射文件：

mmap

与磁盘同步(刷盘)：

fsync, msync

修改权限：

chmod

获取文件信息：

stat



文件系统层 -- page cache

Linux 系统默认的buffered IO读写会“吃掉”暂时空闲的内存空间用于缓存文件数据，这部分缓存称为 page cache。

Page cache 对提升磁盘I/O性能，充分利用内存资源。

但有些应用在用户态实现自己的文件缓存，因此可以用direct IO来绕过page cache。



文件系统层 -- page cache

Buffered I/O (**Page Cache**): 内核以4K页为单位进行IO请求和文件数据缓存。

Direct I/O (**O_DIRECT**): 不使用page cache, 但偏移/地址/大小等要和扇区大小对齐。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5
6 #define FILENAME "test_file"
7
8 int main()
9 {
10     int fd = open(FILENAME, O_CREAT|O_RDWR, 0755);
11     int size = 123;
12     int offset = 456;
13     void *buf = malloc(size);
14     int n = pwrite(fd, buf, size, offset);
15     if (n < 0)
16         return -1;
17     return 0;
18 }
```

Buffered I/O 写示例

```
1 // 需要加入以下宏定义来支持 direct_IO
2 #define _GNU_SOURCE
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <fcntl.h>
7 #include <unistd.h>
8
9 #define FILENAME "test_file"
10 #define SECTOR_SIZE 512
11
12 int main()
13 {
14     int fd = open(FILENAME, O_CREAT|O_RDWR|O_DIRECT, 0755);
15     int n_sector = 5;
16     int off_sector = 3;
17     void *buf;
18     // 申请的buffer的起始地址需要对齐扇区
19     posix_memalign(&buf, SECTOR_SIZE, SECTOR_SIZE * n_sector);
20     // 写文件时, 写数据的大小、偏移都需要对齐扇区。
21     int n = pwrite(fd, buf, SECTOR_SIZE * n_sector, SECTOR_SIZE * off_sector);
22     if (n < 0)
23         return -1;
24     return 0;
25 }
```

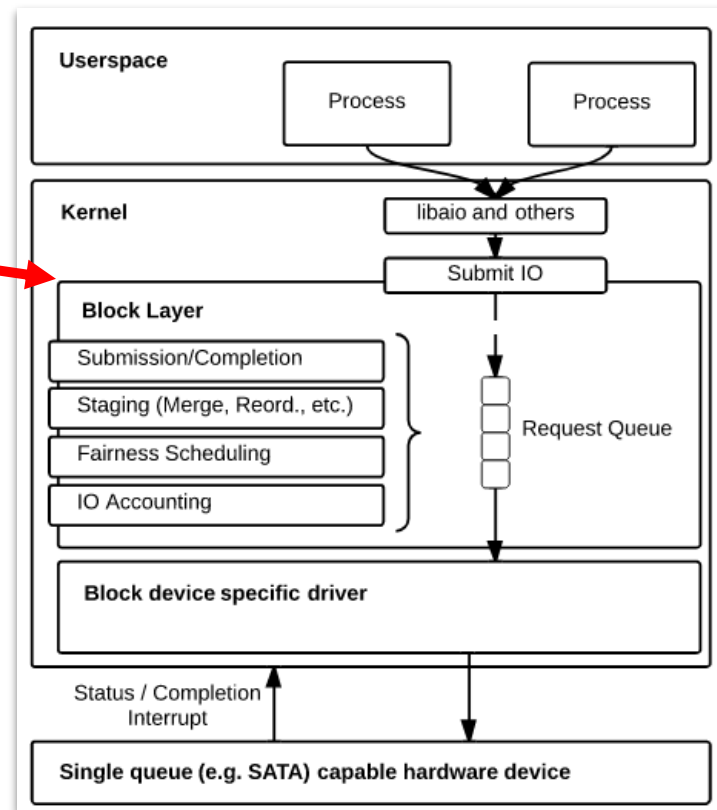
Direct I/O 写示例



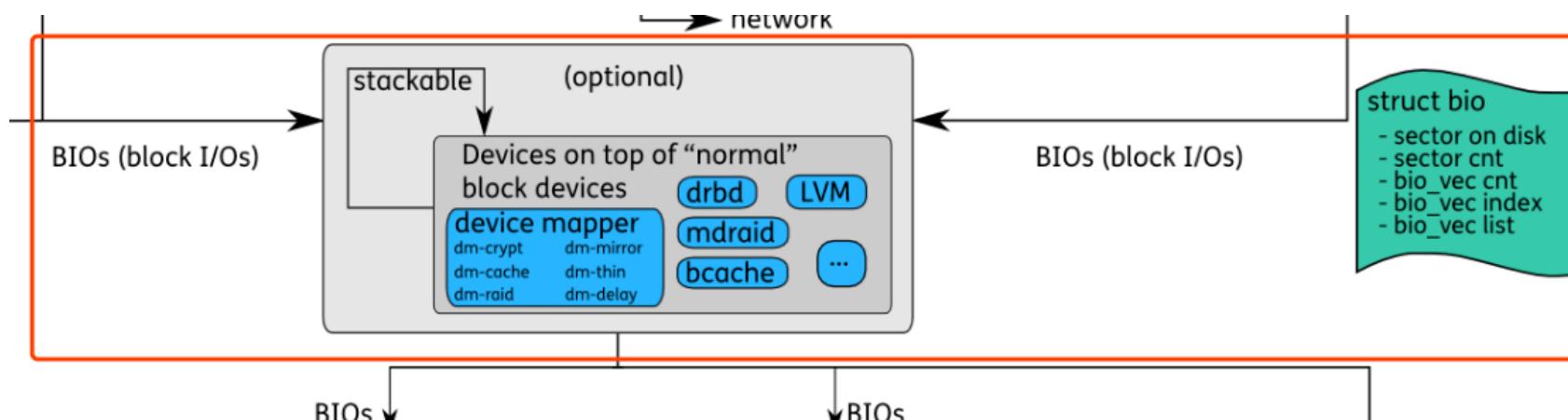


Linux 块层

- 连接上层和设备驱动程序的一接口
- 提供许多常见服务，如IO缓冲、调度、公平性、会计、错误处理等。

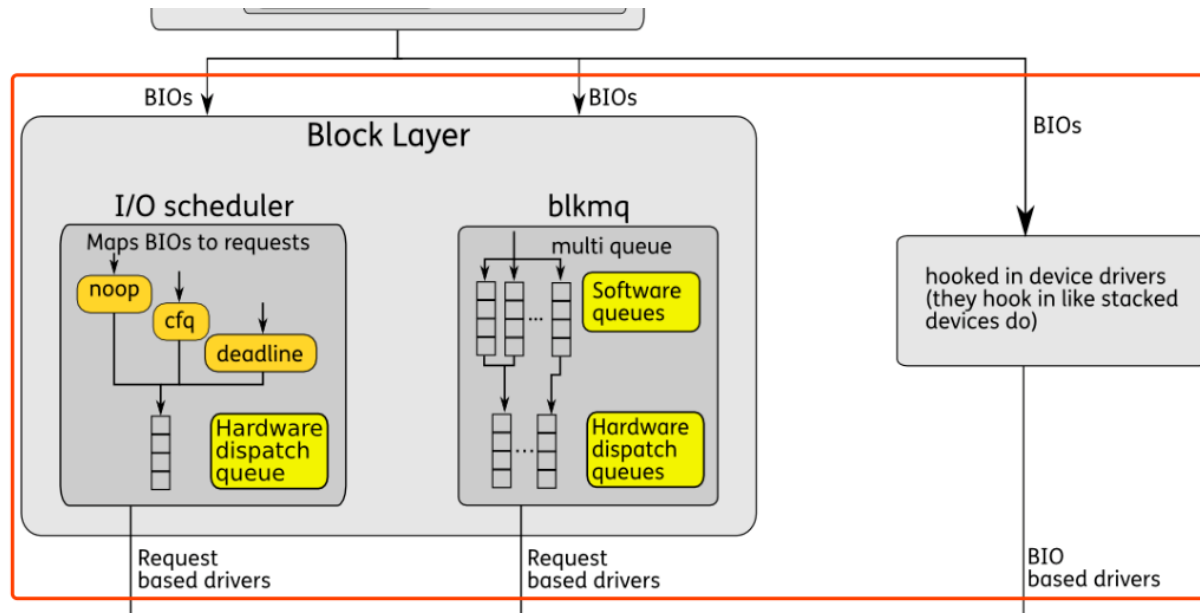


Linux 内核存储栈 -- 块层 Stackable Layers



Stackable block layer (bio layer) 管理逻辑块到物理资源的映射，实现一些存储虚拟化的特性。支持md (RAID), LVM卷, 块层cache(Bcache)等功能。数据重复删除、远程备份等功能也适合在这一层实现。

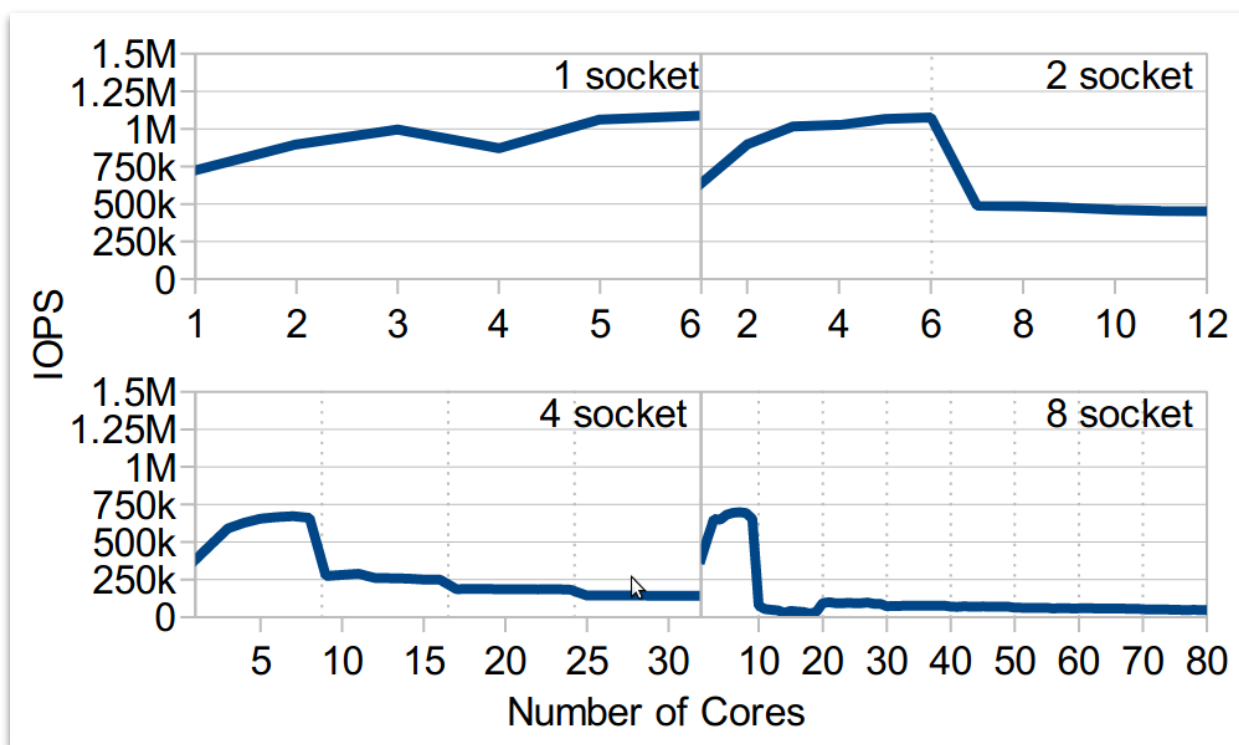
Linux 内核存储栈 -- 块层 I/O Scheduler



I/O调度器:

- noop (no operation): 简单的FIFO，直接插入到调度队列的末尾。
- deadline: 有4个请求队列。其中有一对分别是放读写请求队列(排序队列)；另一对是按最后期限排列的读写请求队列(deadline队列)。
- CFQ(complete fairness queue): 多个请求队列，用hash将一个进程号的请求发到一个请求队列（因此，一个进程常发到一个队列）。
- Anticipatory("预期", 默认算法): 类似deadline，但加入了一些启发式准则。

I/O块层的性能评估

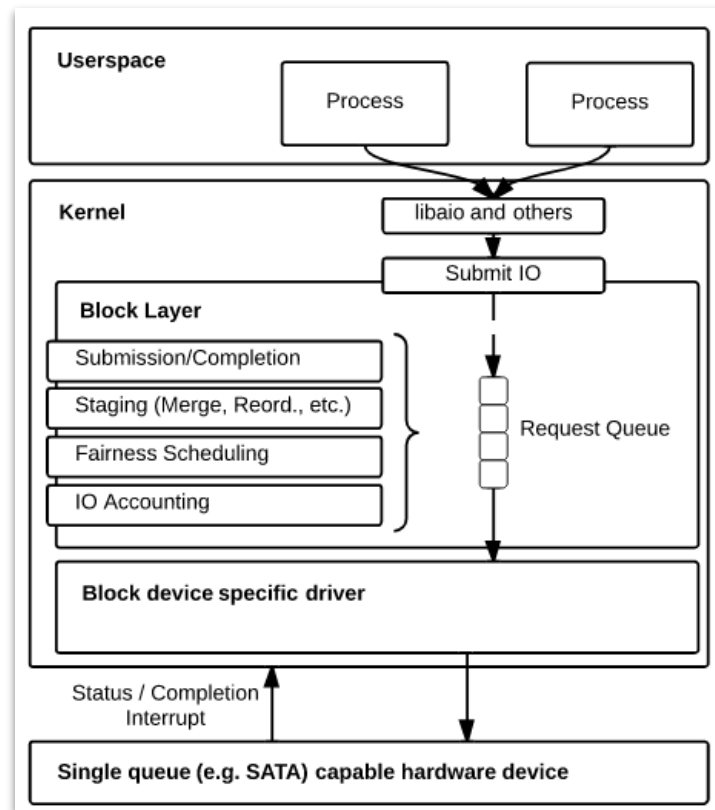


单队列块层，随着核心数量增加，性能会崩溃（猜猜看为什么？）



Linux 块层

- Linux 块式 I/O 层都是单队列设计，CPU多核发来的请求需要排队处理
- 能否充分利用多核CPU环境，并且充分利用底层SSD设备的并行性，扩展I/O的性能？



块层的多队列

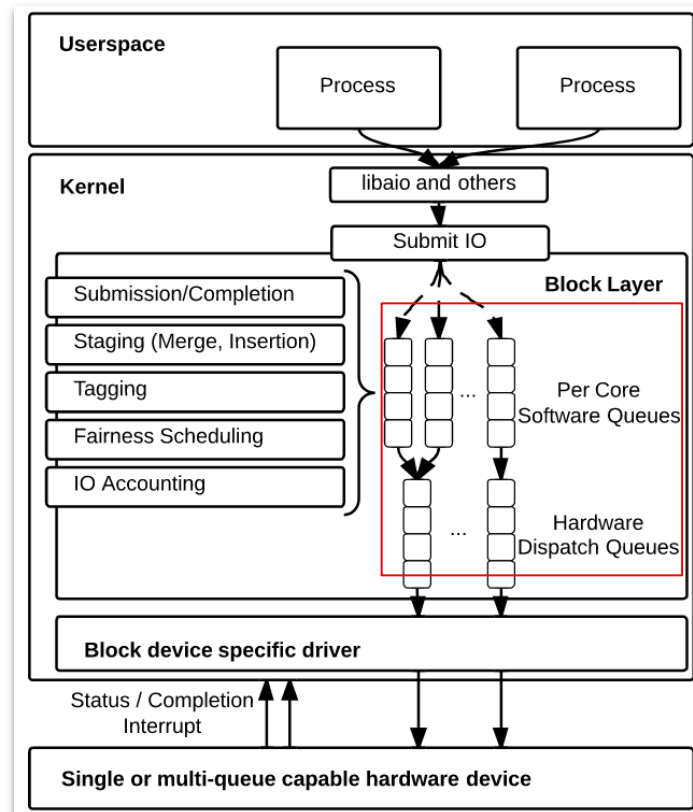
两级分流多队列接口

软件预留队列

- 每个队列连接到一个CPU核心/NUMA节点，减少争用和跨NUMA内存访问
- 软件队列的操作不需要在核心之间产生同步

硬件调度队列

- 设备支持任意数量的队列
- 有助于分发中断





块层

multi-queue的性能表现

- (1) 利用了多核的特性提升IO调度层的可扩展性
- (2) 可以更好地利用NVMe等高性能存储设备的并性能

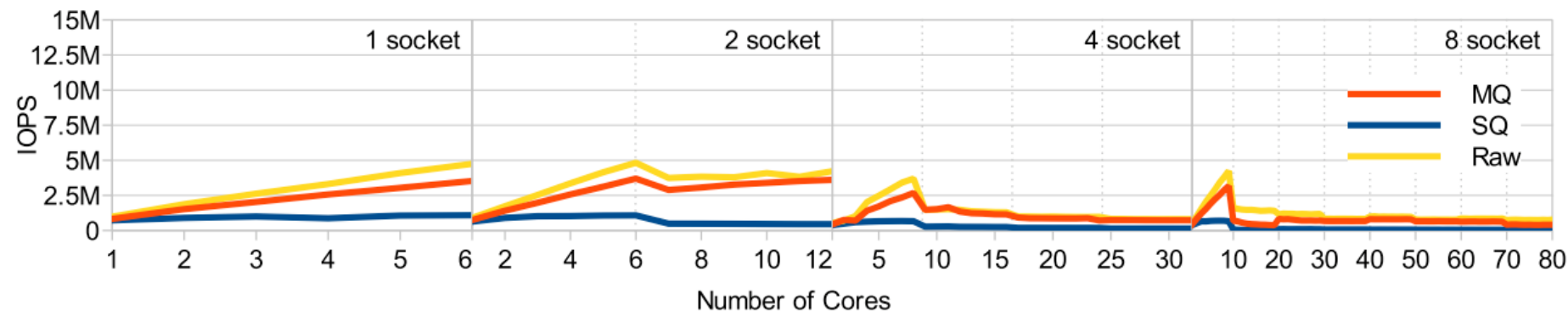


Figure 6: IOPS for single/multi-queue and raw on the 1, 2, 4 and 8-nodes systems.

When Poll is Better than Interrupt (2012)

When Poll is Better than Interrupt

Jisoo Yang

Dave B. Minton

Frank Hady

{jisoo.yang | dave.b.minton | frank.hady} (at) intel.com

Intel Corporation

Abstract

In a traditional block I/O path, the operating system completes virtually all I/Os asynchronously via interrupts. However, performing storage I/O with ultra-low latency devices using next-generation non-volatile memory, it can be shown that polling for the completion – hence wasting clock cycles during the I/O – delivers higher performance than traditional interrupt-driven I/O. This paper thus argues for the *synchronous completion* of block I/O first by presenting strong empirical evidence showing a stack latency advantage, second by delineating limits with the current interrupt-driven path, and third by proving that synchronous completion is indeed safe and correct. This paper further discusses challenges and opportunities introduced by synchronous I/O completion model for both operating system kernels and user applications.

1 Introduction

When an operating system kernel processes a block storage I/O request, the kernel usually submits and completes the I/O request asynchronously, releasing the CPU to perform other tasks while the hardware device completes the storage operation. In addition to the CPU

pass the kernel's heavyweight asynchronous block I/O subsystem, reducing CPU clock cycles needed to process I/Os. However, a necessary condition is that the CPU has to spin-wait for the completion from the device, increasing the cycles used.

Using a prototype DRAM-based storage device to mimic the potential performance of a very fast next-generation SSD, we verified that the synchronous model completes an individual I/O faster and consumes less CPU clock cycles despite having to poll. The device is fast enough that the spinning time is smaller than the overhead of the asynchronous I/O completion model.

Interrupt-driven asynchronous completion introduces additional performance issues when used with very fast SSDs such as our prototype. Asynchronous completion may suffer from lower I/O rates even when scaled to many outstanding I/Os across many threads. We empirically confirmed this with Linux,* and examine the system overheads of interrupt handling, cache pollution, CPU power-state transitions associated with the asynchronous model.

We also demonstrate that the synchronous completion model is correct and simple with respect to maintaining I/O ordering when used with application interfaces such

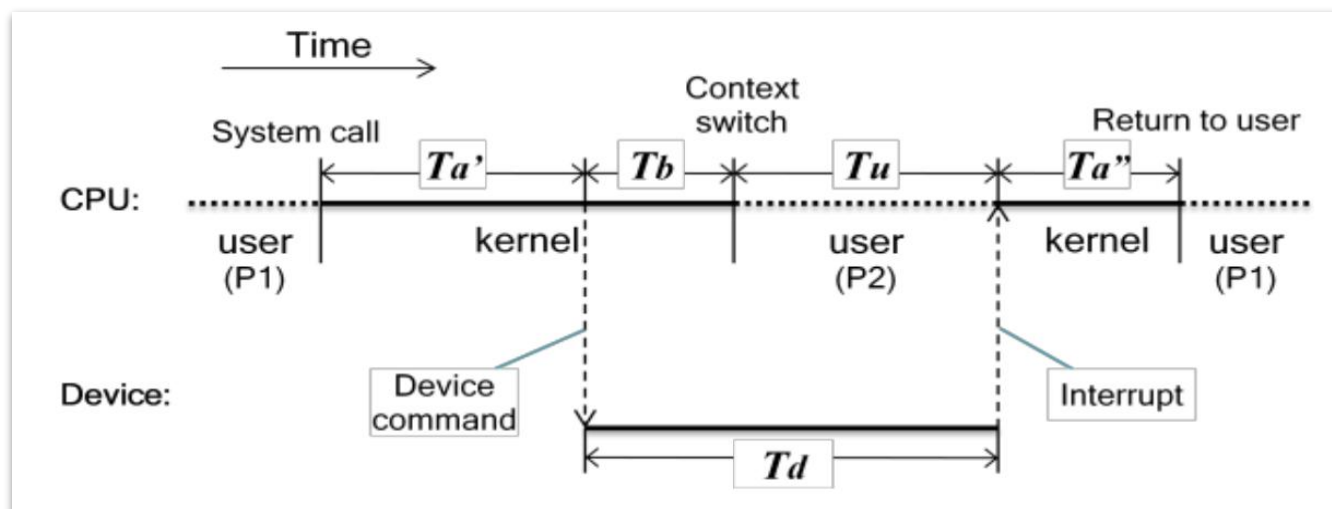
*Any guesses, why?
Faster may be, but better? How?*

经典I/O模式

传统I/O模式

CPU发出I/O请求然后切换处理其他任务

(未来某个时候) 请求完成后, 通过中断通知



https://events.static.linuxfound.org/sites/events/files/slides/lemoal-nvme-polling-vault-2017-final_0.pdf



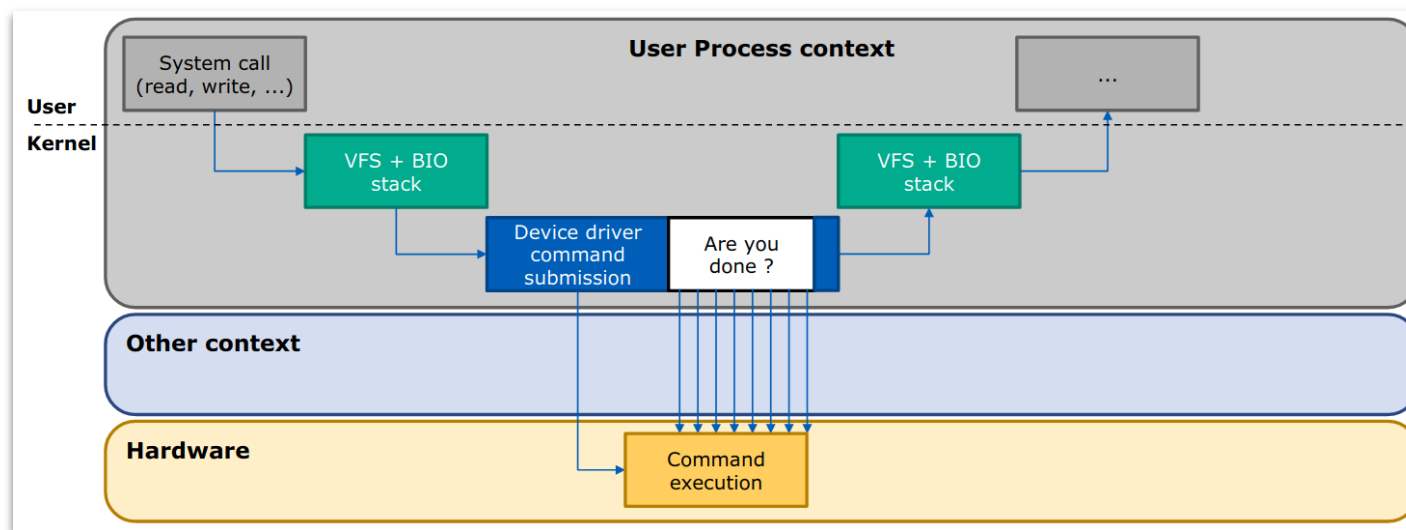
经典I/O方式的挑战

- 设备延迟显著降低
- 延迟降低至几十微秒，开销从硬件转移到软件
- 调度和上下文切换的延迟与设备 I/O 相当
- 在 I/O 即将完成的情况下进行上下文切换是否合理？系统调用呢？
- 中断的生成和处理需要时间
- 中断会破坏当前的执行上下文
- 缓存性能不佳
- “负载间隙” 会导致 CPU 进入节能状态，从而引入 1-2 微秒的延迟

“回退到旧的方案”：轮询



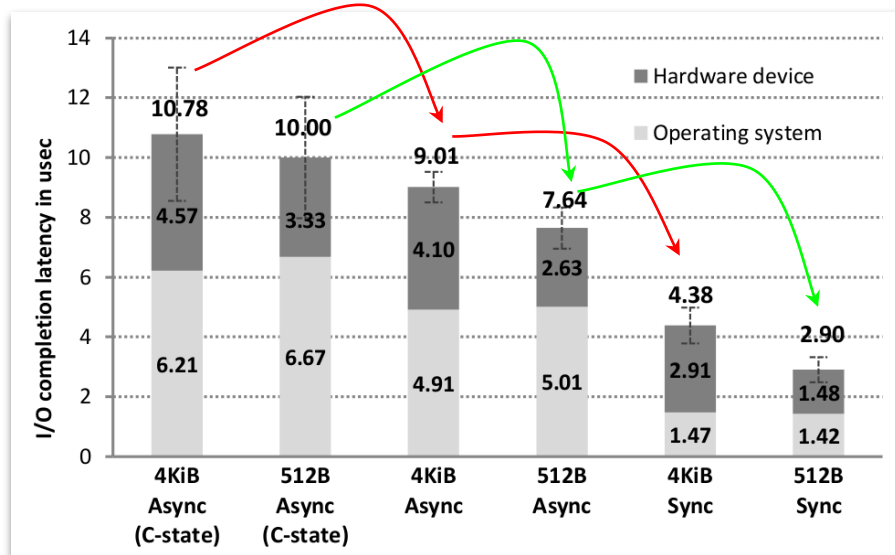
不断轮询I/O请求命令是否完成



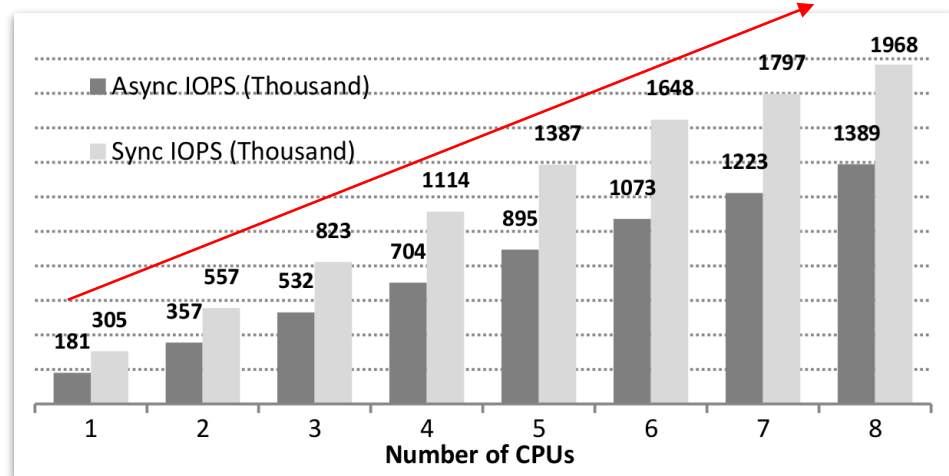
https://events.static.linuxfound.org/sites/events/files/slides/lemoal-nvme-polling-vault-2017-final_0.pdf

性能对比：中断对比轮询

- sync代表轮询，async代表中断



对于单一I/O延迟
CPU陷入节能状态引入了延迟
轮询比中断更快

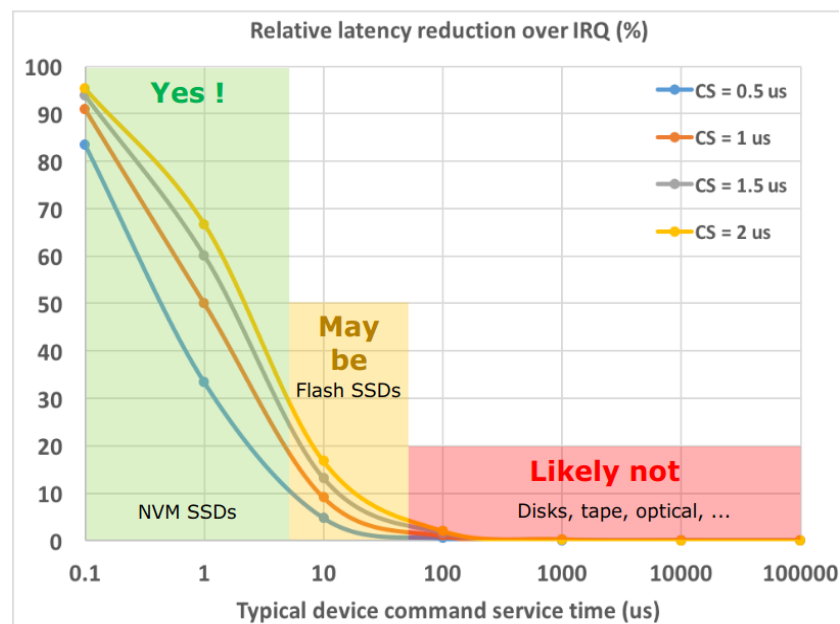


由于不中断，上下文切换减少→导致CPU的利用率更高

何时使用轮询

正如所有系统一样，没有完美的优化，什么时候采用轮询？

- 当设备自身开销与软件开销接近时
 - 软件调度成本
 - 中断成本
 - 设备硬件延迟
- 应用I/O方案的选择 overheads
 - 我们可以依赖内核自身决定何时使用轮询吗？
 - 如果不能依赖内核自己决定，用户是否能告诉内核需要明确采用轮询？
 - 新I/O接口(io_uring)



简短回答：测量然后决定：-)



更为激进的模式-用户态I/O

系统调用，以及内核和用户态之间的数据拷贝也需要时间
既然内核本身造成瓶颈，那就绕过它！

Linux提供UIO 或 VFIO 技术，将设备驱动逻辑从内核态移至用户态。

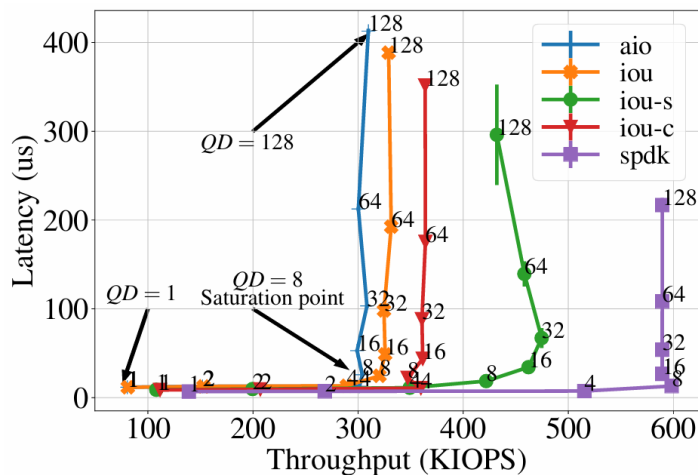
SPDK (Storage Performance Development Kit)

- 高效的用户态I/O库，避免系统调用（内核态用户态切换）
- 轮询I/O
- 减少数据拷贝次数

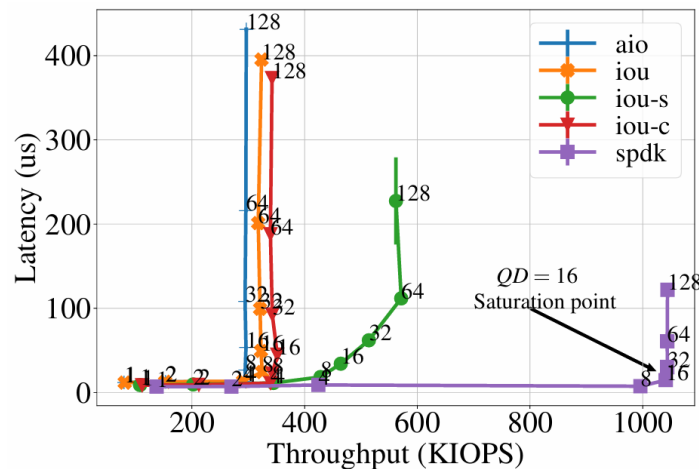


性能对比

- aio: linux AIO
- iou: io_uring 中断模式
- iou-s: io_uring (polled) 普通轮询模式
- iou-c: io_uring (kernel polled) 减少系统调用轮询
- Spdk: 完全用户态轮询I/O



1 Optane SSD



7 Optane SSD



本次课程总结

- 不同存储设备（HDD, SSD等）的硬件原理
- 衡量存储设备性能的指标和方法
- 硬件存储栈的变化
 - 从AHCI/SATA到支持高性能SSD的NVMe协议
- 软件存储栈的变化
 - Linux 存储软件栈
 - 面向多核环境优化的I/O块层多队列
 - 面向高速存储设备的轮询I/O





Thanks! Questions?